

ENSIIE - ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE
POUR L'INDUSTRIE ET L'ENTREPRISE

RAPPORT DE STAGE

PREMIÈRE ANNÉE

Accelerated AI Kernels : Low-Level Optimization of Algorithmic Building Blocks for AI

Auteur :
Tommy MORALES

Sous la direction de :
Shivani RAJPAL (Maître de Stage)
Dimitri WATEL (Tuteur Académique)

Earth Academy
(Amsterdam, Pays-Bas)

Période du 14 Juin 2025 au 25 Août 2025

Remerciements

Je tiens tout d’abord à exprimer ma profonde gratitude envers ma maître de stage, Shivani Rajpal, pour m’avoir accordé sa confiance et offert cette opportunité de mener un projet de recherche ambitieux en autonomie.

Je remercie également Dominic Hillenbrand, Valentin Honoré, et Vincent Fagnon, pour leur disponibilité, leur expertise et leurs conseils avisés qui ont été déterminants dans l’orientation et la réussite de ce projet. Grâce à leurs retours, j’ai eu l’opportunité de transformer ce travail en une contribution scientifique HAL.

Apports personnels

Au-delà des compétences techniques acquises en programmation CUDA bas-niveau et en analyse micro-architecturale, ce stage a été une formation accélérée à la méthodologie de la recherche en optimisation de performance, et une excellente opportunité pour découvrir les bonheurs du benchmarking.

Au-delà des optimisations. Le tournant du stage n’est pas venu d’une analyse complexe de performance, mais d’une question presque naïve : les fragments WMMA manipulent les données directement dans les registres, alors pourquoi toutes les méthodes standards les forcent-elles à faire un aller-retour coûteux par la mémoire partagée juste après ? Cette simple question a changé la nature du projet. Ce qui n’était qu’une tâche d’optimisation est devenu une remise en cause d’une pratique établie. Le reverse-engineering de la structure WMMA n’était plus une simple curiosité, mais une nécessité pour pouvoir explorer une alternative et tenter de supprimer ce détour.

Isoler pour mieux prouver. L’enthousiasme de la découverte s’est vite heurté à la réalité du code. La première tentative d’intégrer l’approche in-register directement dans le kernel V5 (notre version la plus optimisée) a été un échec. Ce kernel était si finement réglé, poussant le matériel à ses limites, que le moindre changement avait des effets de bord imprévisibles. Dans un tel environnement, où chaque optimisation dépend des autres, il était impossible d’isoler l’impact réel de notre nouvelle méthode.

Cet échec a forcé une remise en question de la méthode. Plutôt que de s’acharner sur un code trop complexe, la décision a été prise de repartir de zéro, ou presque. Deux nouveaux kernels de test ont été développés sur une base commune et simplifiée. Le premier implémentait la méthode standard, le second notre approche in-register. Cette comparaison contrôlée a permis de démontrer sans ambiguïté la supériorité de la nouvelle technique, même si les performances brutes de ces kernels de test restaient modestes. La leçon fut claire : pour valider une idée nouvelle, il est parfois nécessaire de sacrifier la performance maximale pour garantir la rigueur et la fiabilité de la mesure.

La permutation des données : une leçon de pragmatisme. La conception de l’algorithme de permutation a résulté d’un processus d’élimination méthodique. Une approche naïve, qui aurait conservé la structure des données, menait à des schémas d’accès aux registres inexploitable. À l’opposé, l’option d’une réorganisation complète vers un layout linéaire parfait s’est heurtée à un constat simple : le coût en instructions shuffle aurait largement dépassé les bénéfices. La solution viable se trouvait donc dans un juste milieu : une permutation partielle. L’objectif n’était pas la perfection structurelle, mais l’efficacité pratique : regrouper les données juste assez pour que la réduction qui suivait soit la plus rapide possible. Ce fut la confirmation que l’optimisation la plus efficace est souvent un compromis bien pensé, plutôt qu’une solution théorique absolue.

Synthèse

Ce stage, mené dans le cadre de la mission de Green IT de l'entreprise Earth Academy, visait à explorer l'optimisation logicielle comme levier pour un numérique plus durable. Le mandat initial était d'analyser les mécanismes d'attention, au cœur des IA modernes, afin de quantifier le compromis entre leur performance (latence) et leur consommation d'énergie.

Pour mener cette analyse, nous avons d'abord construit un cadre expérimental rigoureux à partir de zéro. En nous basant uniquement sur la description théorique des algorithmes, nous avons implémenté notre propre suite de "kernels" (programmes GPU). Cette suite commence par une version naïve de l'attention, puis inclut notre propre implémentation de FlashAttention, et enfin intègre progressivement des optimisations micro-architecturales de plus en plus fines. Un protocole de benchmark automatisé nous a permis de mesurer objectivement l'impact de chaque technique, en construisant des frontières de Pareto pour visualiser le compromis latence-énergie.

Cette analyse systématique a révélé un goulot d'étranglement fondamental, persistant même dans les approches les plus optimisées. Après le calcul matriciel principal, nous avons observé un aller-retour coûteux et systématique des données entre les registres (la mémoire la plus rapide) et la mémoire partagée. Cette découverte a réorienté la phase finale du stage vers une solution radicale : la suppression complète de ce transit. En nous basant sur du reverse-engineering de la structure interne des données WMMA, nous avons conçu un algorithme de permutation inédit. Cette technique permet de réorganiser les données directement dans les registres pour effectuer une réduction parallèle sans aucun accès à la mémoire partagée.

Intégrée à un kernel complet, notre approche a démontré un gain de performance de 1.40x, validant son potentiel et dépassant les objectifs initiaux du stage.

Mots-clés : GPU, CUDA, Optimisation de kernels, FlashAttention, Efficacité Énergétique, Calcul Haute Performance (HPC), Benchmark, Analyse de Performance, Frontière de Pareto, WMMA, Micro-architecture.

Table des matières

1	Introduction	4
1.1	Contexte : L'IA face au défi écologique	4
1.2	Problématique et Démarche : Cartographier les nouveaux goulots d'étranglement	4
2	État de l'Art : L'Ascension des Transformers et la Quête de l'Efficacité	5
2.1	Le Contexte : L'Ère des Grands Modèles de Langage (LLMs)	5
2.2	Le Mécanisme d'Attention : Une Approche Intuitive	5
2.3	De l'Élégance Conceptuelle au Mur Computationnel	6
2.4	FlashAttention : La Révolution I/O-Aware	6
2.5	L'Après-FlashAttention : Le Nouveau Front d'Optimisation	7
3	Contexte Technique et Outils	7
3.1	Du CPU au GPU : un changement de paradigme architectural	7
3.2	Outils et environnement de travail	8
4	Travaux Réalisés : Une Démarche d'Optimisation Incrémentale	9
4.1	Cadre Expérimental et Protocole de Mesure	9
4.2	Développement et Analyse de la Suite de Kernels	9
4.2.1	Version 0 - Baseline Naïve (Référence Globale)	9
4.2.2	Version 1 - Kernel Fusion et Softmax Itératif	11
4.2.3	Version 2 - Optimisation des Accès par Coalescence	14
4.2.4	Version 3 - Optimisation de la Réduction par Primitives Intra-Warp	17
4.2.5	Version 4 - Masquage de la Latence par Double Buffering (K et V)	19
4.2.6	Version 5 - Accélération Matérielle via les Tensor Cores (WMMA)	22
4.2.7	Résultats	25
4.2.8	Le Pivot : Synthèse des Résultats et Identification du Goulot Résiduel . .	25
4.3	Reverse-Engineering du Layout WMMA	25
4.4	Stratégie de Réorganisation des Données : La Permutation Partielle	27
4.5	Exécution : Réduction Parallèle In-Register	28
4.6	Intégration dans un Cas d'Usage : Le Softmax de FlashAttention	29
5	Résultats Obtenus	30
5.1	Protocole Expérimental	30
5.2	Analyse de la Démarche d'Optimisation Incrémentale	31
5.3	Validation de l'Approche In-Register	33
6	Conclusion et Perspectives	34
6.1	Conclusion	34
6.2	Limites et Perspectives	34
A	Développement Durable et Responsabilité Sociétale	36
A.1	Pressions sur l'Environnement	36
A.2	Politique, Stratégie et Actions pour Minimiser les Impacts	37
B	Preuve Qualitative de la Sous-Optimalité des Réorganisations Intuitives	38
C	Preuve de l'Algorithme de Softmax Itératif	38
D	Code source des kernels du micro-benchmark	39
E	Code Source du Kernel Final In-Register	42

1 Introduction

1.1 Contexte : L'IA face au défi écologique

L'intelligence artificielle, notamment les grands modèles de langage basés sur l'architecture Transformer [1], est en train de transformer notre monde. Cependant, cette révolution a un coût caché : une consommation d'énergie colossale. Entraîner et utiliser ces modèles demande une puissance de calcul si intense que les centres de données qui les hébergent consomment autant d'électricité que des pays entiers, un enjeu majeur pour la recherche sur l'IA [2, 3].

C'est dans ce contexte que s'inscrit ce stage chez **Earth Academy**, une entreprise dont la mission est d'aider les organisations à réduire leur impact environnemental, en particulier celui lié au numérique (le **Green IT**). Pour Earth Academy, la question est stratégique : au lieu de simplement conseiller de moins utiliser l'IA, ne serait-il pas possible de rendre l'IA elle-même radicalement plus efficace, en accord avec les principes du Green AI [3] ?

L'intérêt de ce stage pour l'entreprise était donc de mener un projet exploratoire : prouver qu'en optimisant le **moteur** de l'IA au plus bas niveau, on peut obtenir des gains de performance et d'efficacité énergétique spectaculaires. Ce travail visait à poser les bases d'une future offre de conseil technique, où l'optimisation logicielle devient un levier majeur de la transition écologique.

1.2 Problématique et Démarche : Cartographier les nouveaux goulots d'étranglement

Pour comprendre notre démarche, imaginons que le processeur d'un ordinateur (un GPU) est une immense usine. Les calculs de l'IA nécessitent de faire circuler d'énormes quantités de données entre un entrepôt distant (la mémoire principale) et les chaînes de montage de l'usine (les cœurs de calcul).

Pendant longtemps, le principal problème était un **énorme embouteillage sur l'autoroute** entre l'entrepôt et l'usine. Des algorithmes récents, comme le fameux *FlashAttention* [4, 5], ont agi comme un TGV, fluidifiant radicalement ce trafic.

Cependant, cette révolution a déplacé le problème. L'autoroute est dégagée, mais des inefficacités et des goulots d'étranglement plus subtils persistent **à l'intérieur de l'usine elle-même**. Chaque technique d'optimisation (mieux organiser les palettes, préparer le travail en amont, utiliser des machines spécialisées) a un impact sur deux facteurs parfois contradictoires : la **vitesse de production** (la performance) et la **consommation d'énergie** de l'usine (l'efficacité énergétique).

La problématique centrale de ce stage est donc la suivante : une fois le goulot d'étranglement majeur résolu, où se situent les nouvelles opportunités d'optimisation, et comment quantifier précisément leur impact sur le couple performance-efficacité énergétique ?

Cette analyse devait permettre à Earth Academy de démontrer par la preuve la valeur de l'optimisation bas-niveau, en vue de structurer une future offre de conseil.

Pour répondre à cette problématique, notre démarche a suivi les missions définies :

1. **Construire un laboratoire de mesure fiable** : La première étape a consisté à étudier l'état de l'art et à développer une série de programmes (des **kernels**), allant d'une version de base à des versions de plus en plus sophistiquées. En parallèle, nous avons mis en place un protocole de *benchmarking* robuste et automatisé pour mesurer avec précision la latence et la consommation d'énergie de chaque version, sur du matériel de pointe via un service de cloud computing.
2. **Identifier et résoudre les goulots d'étranglement un par un** : Grâce à ce laboratoire, nous avons mené une investigation systématique. L'analyse de chaque version révélait un nouveau goulot d'étranglement (un chargement de données inefficace, une synchronisation trop coûteuse...). Nous implémentions alors une technique d'optimisation ciblée et mesurions

son gain. Cette chasse aux goulots d'étranglement nous a menés à identifier un obstacle résiduel fondamental, même dans les versions les plus optimisées : un aller-retour coûteux des données vers un quai de transit interne (la mémoire partagée).

3. **Explorer une solution de rupture** : L'identification de ce dernier goulot a ouvert la phase la plus exploratoire du stage. Nous nous sommes posé une question plus audacieuse : est-il possible d'éliminer *entièrement* ce transit ? Cette piste nous a conduits à concevoir une nouvelle technique d'optimisation radicale, en reconfigurant les flux de données directement sur les chaînes de montage (les registres). Cette contribution, au cœur de ce rapport, a été formalisée dans une pré-publication scientifique [6].

Ce rapport documente donc cette démarche complète : de la mise en place d'un cadre expérimental rigoureux à la découverte et la validation d'une innovation, en passant par l'analyse quantifiée de l'état de l'art.

2 État de l'Art : L'Ascension des Transformers et la Quête de l'Efficacité

Pour saisir la pertinence de l'optimisation micro-architecturale, il est indispensable de comprendre les fondements des modèles de langage modernes, la nature de leur goulot d'étranglement historique, et comment sa résolution a ouvert un nouveau front d'optimisation.

2.1 Le Contexte : L'Ère des Grands Modèles de Langage (LLMs)

Au cœur de la révolution actuelle de l'IA se trouvent les **Grands Modèles de Langage (LLMs)**. Fondamentalement, un LLM est un réseau de neurones massif, entraîné sur des quantités colossales de texte, dont l'objectif principal est de maîtriser la complexité du langage humain. Leur utilité est immense : ils peuvent résumer des documents, traduire des langues, écrire du code, répondre à des questions complexes, et même tenir une conversation cohérente. Nous en avons besoin car ils permettent d'automatiser et d'assister des tâches intellectuelles qui étaient jusqu'alors hors de portée des machines.

L'architecture qui a rendu ces LLMs possibles est le **Transformer** [1]. Avant son arrivée, les modèles de langage (comme les RNNs) traitaient le texte de manière séquentielle, mot après mot. C'était lent et limitait leur capacité à saisir des dépendances à longue distance dans un texte. Le Transformer a introduit une innovation majeure : la capacité de traiter tous les mots d'une phrase simultanément, en parallèle, grâce à un mécanisme central appelé **l'attention**.

2.2 Le Mécanisme d'Attention : Une Approche Intuitive

L'attention est la "sauce secrète" du Transformer. Plutôt que de simplement lire les mots dans l'ordre, elle permet au modèle, pour chaque mot, de "regarder" tous les autres mots de la séquence et de décider lesquels sont les plus importants pour comprendre le contexte.

Pour rendre cela concret, imaginons que le modèle traite la phrase : "Le robot a ramassé l'avocat car il était mûr". Pour comprendre à quoi se réfère "il", le modèle doit porter son attention sur "avocat" et non sur "robot". Le mécanisme d'attention est ce qui lui permet d'apprendre à faire ce lien.

Ce processus est formalisé par une équation qui, bien qu'intimidante, peut être comprise par une analogie avec une recherche d'information :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Décomposons cette formule en étapes logiques :

1. **La Requête, les Clés, les Valeurs (Q, K, V)** : Pour chaque mot, on crée trois vecteurs : une **Requête** (Query), une **Clé** (Key) et une **Valeur** (Value).
 - La **Query** représente la question : "Pour comprendre ce mot, quelle information je recherche?".
 - La **Key** est comme l'étiquette ou le mot-clé de chaque mot : "Voici le type d'information que je contiens".
 - La **Value** est le contenu réel du mot : "Voici l'information que je peux fournir".
2. **Calcul des Scores de Similarité (QK^T)** : Pour chaque mot, on compare sa *Query* à la *Key* de tous les autres mots de la séquence. Cette opération (un produit scalaire) produit un score élevé si la Requête et la Clé sont similaires. On obtient ainsi une matrice S où S_{ij} mesure "l'importance" du mot j pour la compréhension du mot i .
3. **Normalisation des Scores (softmax)** : Les scores bruts sont ensuite passés à travers une fonction *softmax*. Son rôle est de transformer ces scores en une distribution de probabilités : une série de poids entre 0 et 1 dont la somme est égale à 1. Ces poids représentent le pourcentage d'attention que le mot i doit accorder à chaque mot j .
4. **Synthèse Pondérée ($\dots V$)** : Enfin, pour créer la nouvelle représentation du mot i , on calcule une somme pondérée des *Values* de tous les mots de la séquence, en utilisant les poids d'attention calculés à l'étape précédente. Ainsi, les mots jugés les plus pertinents (ceux avec un poids d'attention élevé) contribuent le plus à la nouvelle représentation.

Pourquoi la différentiabilité est-elle cruciale ? Toutes ces opérations (multiplication de matrices, softmax) sont **différentiables**. C'est une propriété mathématique essentielle. Pour qu'un modèle puisse *apprendre*, il doit pouvoir ajuster ses paramètres internes pour minimiser ses erreurs. La méthode standard pour ce faire est la descente de gradient, qui nécessite de calculer la "pente" de l'erreur par rapport à chaque paramètre. La différentiabilité est ce qui nous permet de calculer cette pente et donc de "remonter" l'erreur à travers le réseau pour qu'il s'améliore. Sans elle, l'entraînement du Transformer serait impossible.

2.3 De l'Élégance Conceptuelle au Mur Computationnel

Cette architecture, bien que puissante, a un coût. Le calcul de la matrice de scores S (QK^T) a une complexité en $O(N^2)$, où N est la longueur de la séquence. Pour une séquence de 4096 mots, cela représente une matrice de plus de 16 millions d'éléments. Le goulot d'étranglement n'est pas tant le nombre d'opérations que la mémoire.

Cette matrice intermédiaire est trop grande pour tenir dans la mémoire rapide du GPU (SRAM, quelques mégaoctets). L'implémentation standard est donc forcée d'effectuer un aller-retour coûteux : 1. Calculer un bloc de S sur le GPU. 2. **Écrire ce bloc dans la lente mémoire globale (HBM, plusieurs gigaoctets)**. 3. Répéter jusqu'à ce que S soit entièrement stockée en HBM. 4. **Relire la matrice S complète depuis la HBM** pour calculer le softmax. 5. **Relire encore le résultat du softmax depuis la HBM** pour le multiplier par V .

Ces transferts massifs et redondants avec la HBM sont des centaines de fois plus lents que les opérations en SRAM. C'était le principal obstacle à l'efficacité des Transformers.

2.4 FlashAttention : La Révolution I/O-Aware

FlashAttention [4, 5] a été conçu pour briser ce mur. Son principe est simple mais révolutionnaire : **ne jamais écrire la matrice d'attention complète en HBM**. En combinant deux techniques :

- **Le Tiling** : Découper les matrices Q , K et V en blocs suffisamment petits pour tenir en SRAM.

- **Le Softmax Itératif** : Un algorithme qui permet de calculer le softmax bloc par bloc, en mettant à jour les statistiques (le maximum et la somme) au fur et à mesure, sans avoir besoin de voir la matrice entière (voir Annexe C).

FlashAttention fusionne toutes les étapes de l'attention en une seule passe. Il charge un bloc de Q, K, V en SRAM, effectue tous les calculs (produit matriciel, softmax, multiplication par V) localement, puis écrit le résultat final directement en HBM, en évitant tout stockage intermédiaire.

2.5 L'Après-FlashAttention : Le Nouveau Front d'Optimisation

En résolvant le goulot d'étranglement de la HBM, FlashAttention a transformé le problème. L'attention n'est plus limitée par la mémoire (*memory-bound*), mais par la puissance de calcul (*compute-bound*). La performance est désormais dictée par l'efficacité des opérations qui se déroulent **à l'intérieur de la puce (on-chip)**.

C'est précisément là que notre travail commence. Maintenant que les allers-retours avec la mémoire externe sont éliminés, le nouveau goulot d'étranglement est devenu plus subtil : c'est le transit des données entre les différents niveaux de la hiérarchie mémoire interne du GPU (mémoire partagée et registres). La quête de performance nous amène donc à nous poser la question suivante : est-il possible d'éliminer même ces derniers mouvements de données on-chip, en confinant le calcul au plus près des unités arithmétiques ?

3 Contexte Technique et Outils

3.1 Du CPU au GPU : un changement de paradigme architectural

La quasi-totalité des calculs en intelligence artificielle moderne repose sur des processeurs graphiques (GPU), dont l'architecture est fondamentalement différente de celle des processeurs centraux (CPU). Alors que les CPU sont optimisés pour minimiser la latence sur des tâches séquentielles complexes, les GPU sont des architectures massivement parallèles conçues pour maximiser le débit sur des opérations simples et répétitives. Cette philosophie est particulièrement adaptée à l'algèbre linéaire, qui constitue le socle mathématique des réseaux de neurones.

Cependant, la performance d'un kernel sur GPU est moins dictée par la puissance de calcul brute (FLOPs) que par la gestion des mouvements de données. L'efficacité d'un algorithme est directement contrainte par sa capacité à naviguer la hiérarchie mémoire du processeur. Comme l'illustre la Figure 1 pour l'architecture NVIDIA H100, cette hiérarchie est marquée par des ordres de grandeur de latence très différents.

Memory in the H100 GPU Architecture

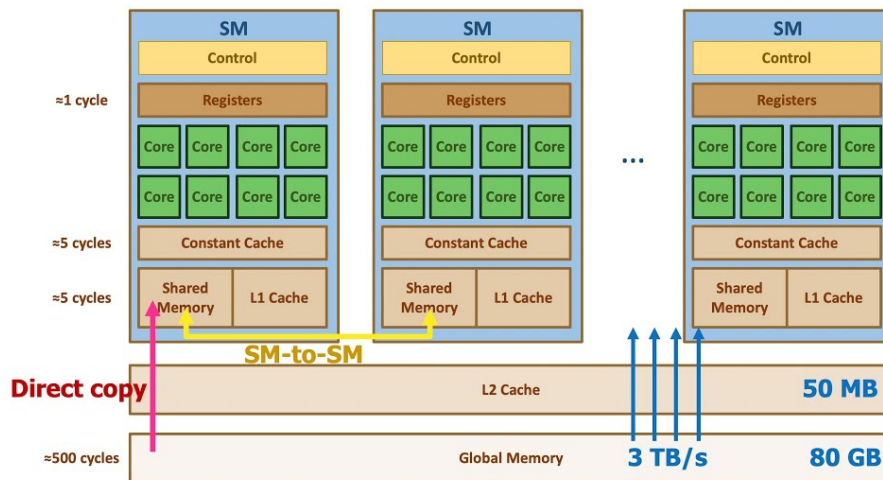


FIGURE 1 – Hiérarchie mémoire de l’architecture NVIDIA H100, illustrant la latence d’accès relative entre les différents niveaux. Source : NVIDIA.

L’analyse de cette architecture, dont les détails sont disponibles dans la documentation NVIDIA [7], révèle plusieurs points critiques :

- L’accès aux **Registres**, la mémoire la plus proche des cœurs de calcul, s’effectue en ≈ 1 cycle.
- L’accès à la **Mémoire Partagée (Shared Memory)**, partagée au sein d’un même Streaming Multiprocessor (SM), ne prend que ≈ 5 cycles. Elle est essentielle pour la collaboration et l’échange rapide de données entre les threads d’un même bloc.
- En revanche, l’accès à la **Mémoire Globale (HBM)**, bien que dotée d’une bande passante très élevée (3 TB/s), impose une latence de ≈ 500 cycles.

Cet écart de latence de près de deux ordres de grandeur entre la mémoire on-chip (Registres, Partagée) et la mémoire off-chip (Globale) est le principal goulot d’étranglement. La programmation GPU à haute performance est donc un exercice de *data locality* : il faut concevoir des algorithmes qui minimisent à tout prix les transferts depuis et vers la mémoire globale, en maximisant la résidence et la réutilisation des données dans les registres et la mémoire partagée.

C’est cette contrainte fondamentale qui a motivé les travaux de ce stage. Après que les algorithmes comme FlashAttention ont optimisé les accès à la mémoire globale, notre investigation s’est concentrée sur le dernier maillon de la chaîne : l’élimination des transferts résiduels entre la mémoire partagée et les registres, afin de confiner le calcul dans le niveau le plus rapide de la hiérarchie mémoire.

3.2 Outils et environnement de travail

Le projet a été mené en s’appuyant sur un ensemble d’outils standards pour le calcul haute performance et l’analyse micro-architecturale. L’environnement expérimental était composé des éléments suivants :

- **Environnement Matériel :** Les développements et les tests ont été effectués sur deux architectures NVIDIA de pointe. La plateforme principale était une carte grand public **RTX 5080** (architecture Blackwell), complétée par des tests de validation sur une carte de centre de données **H100** (architecture Hopper), accessible via le service de cloud computing

LambdaGPU.

- **Langages de Programmation :**
 - **C++/CUDA :** Langage principal pour le développement des kernels GPU, permettant un contrôle de bas niveau sur le matériel.
 - **Python :** Utilisé pour les scripts d'analyse des données de benchmark et la génération des visualisations (e.g., frontières de Pareto).
 - **Bash :** Employé pour l'automatisation et l'orchestration de l'ensemble de la suite de tests.
- **Chaîne de Compilation et Outils de Profilage :**
 - **nvcc :** Le compilateur de l'écosystème CUDA de NVIDIA, utilisé pour transformer le code source en exécutables GPU.
 - **nvidia-smi :** Outil en ligne de commande pour le monitoring en temps réel de l'état des GPU, notamment la consommation énergétique, essentielle à l'analyse de l'efficacité.
 - **Nsight Compute (ncu) :** Le profileur micro-architectural de NVIDIA. Cet outil a été indispensable pour analyser en détail la performance des kernels, quantifier la latence des instructions, et identifier précisément les goulots d'étranglement on-chip.
- **Gestion de Projet et Reproductibilité :** Un **Makefile** a été développé pour automatiser entièrement le cycle de vie des expériences (compilation des différents kernels, exécution des benchmarks, collecte des résultats). Cette approche garantit une reproductibilité totale des résultats présentés dans ce rapport.

4 Travaux Réalisés : Une Démarche d'Optimisation Incrémentale

L'approche adoptée pour ce projet a été celle d'une investigation expérimentale systématique. Partant d'un point de référence minimal, une suite de plus de dix kernels a été développée. Chaque nouvelle version n'introduit qu'une seule technique d'optimisation supplémentaire par rapport à la précédente. Cette méthode permet d'isoler et de quantifier rigoureusement l'impact de chaque décision architecturale sur la performance et l'efficacité énergétique.

4.1 Cadre Expérimental et Protocole de Mesure

Pour garantir la comparabilité et la fiabilité des résultats à travers toutes les versions de kernels, un protocole de mesure strict et automatisé a été mis en place.

- **Mesure de latence :** La performance est mesurée en moyennant la durée d'exécution de plusieurs milliers de lancements d'un même kernel. Une phase de chauffe ("warm-up") de plusieurs centaines de lancements précède systématiquement la prise de mesure pour stabiliser la fréquence du GPU et l'état des caches. Les mesures sont effectuées à l'aide de l'API CUDA Events pour une précision maximale.
- **Mesure énergétique :** La consommation est mesurée par échantillonnage à haute fréquence (10 ms) via **nvidia-smi** pendant une exécution prolongée du kernel. La valeur moyenne sur cette période stable est retenue comme la consommation de référence pour le kernel testé.

Ce protocole est appliqué de manière identique à chaque version du kernel décrite ci-dessous.

4.2 Développement et Analyse de la Suite de Kernels

4.2.1 Version 0 - Baseline Naïve (Référence Globale)

Ce premier kernel implémente l'algorithme d'attention de la manière la plus directe et littérale possible, en traduisant directement les équations mathématiques. Chaque thread est responsable du calcul d'un unique élément de la matrice de sortie `out[i, j]`.

Cette approche présente une inefficacité majeure et volontaire : pour chaque thread, le calcul des scores d'attention (la multiplication matricielle $S = QK^T$) est entièrement recalculé trois fois dans trois boucles successives : une première fois pour trouver la valeur maximale de la ligne, une seconde pour calculer la somme des exponentielles (le dénominateur du softmax), et une troisième pour calculer la pondération finale. Tous les accès se font systématiquement depuis et vers la mémoire globale, ce qui engendre un trafic mémoire colossal et redondant. Ce kernel sert de performance plancher absolue, un "pire cas" qui justifie l'ensemble des optimisations ultérieures.

```
__global__ void naive_attention(float* Q, float* K, float* V, float* out,
                               int seq_len, int d_model) {
    // Chaque thread calcule un élément out[i, j]
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i >= seq_len || j >= d_model) return;

    float sum = 0.0f;
    float max_score = -__FLT_MAX__; // Initialisation correcte

    // --- 1ère passe sur QK^T : Calcul du max pour la stabilité numérique ---
    for (int k = 0; k < seq_len; k++) {
        float score = 0.0f;
        for (int d = 0; d < d_model; d++) {
            score += Q[i * d_model + d] * K[k * d_model + d];
        }
        max_score = fmaxf(max_score, score);
    }

    // --- 2ème passe sur QK^T : Calcul du dénominateur du softmax ---
    float softmax_sum = 0.0f;
    for (int k = 0; k < seq_len; k++) {
        float score = 0.0f;
        for (int d = 0; d < d_model; d++) {
            score += Q[i * d_model + d] * K[k * d_model + d];
        }
        softmax_sum += expf(score - max_score);
    }

    // --- 3ème passe sur QK^T et V : Calcul de la somme pondérée ---
    for (int k = 0; k < seq_len; k++) {
        float score = 0.0f;
        for (int d = 0; d < d_model; d++) {
            score += Q[i * d_model + d] * K[k * d_model + d];
        }
        float attention_weight = expf(score - max_score) / softmax_sum;
        // Chaque thread lit une colonne de V
        sum += attention_weight * V[k * d_model + j];
    }

    out[i * d_model + j] = sum;
}
```

Listing 1 – Structure de calcul du kernel V0. Le calcul des scores (produit scalaire) est répété trois fois, engendrant des lectures redondantes depuis la mémoire globale.

4.2.2 Version 1 - Kernel Fusion et Softmax Itératif

Cette première optimisation s'attaque à l'inefficacité fondamentale de la version naïve : la redondance des calculs et des accès mémoire. L'approche consiste à **fusionner** les trois passes de l'algorithme d'attention en une seule, en implémentant le principe du softmax "itératif" ou "online".

Principe de l'Algorithme Plutôt que de calculer et stocker la matrice de scores S en entier, on la parcourt par blocs (ou "tuiles"). L'intuition fondamentale est de traiter les données en un seul passage, en **corrigeant les statistiques accumulées** au fur et à mesure que de nouvelles informations arrivent. Ce mécanisme de mise à jour corrective garantit l'exactitude mathématique du résultat final. Une preuve formelle par récurrence est fournie en Annexe C.

Implémentation du Kernel V1 Le code ci-dessous met en œuvre cette fusion. Après avoir calculé une tuile de la matrice de scores sS en mémoire partagée, une réduction parallèle est effectuée pour trouver les statistiques du softmax.

La Réduction Parallèle en Mémoire Partagée Pour agréger rapidement les valeurs au sein d'une ligne de la tuile (par exemple, pour trouver le maximum), on utilise une technique de **réduction en arbre** via la mémoire partagée. Le principe est illustré par la Figure 2. À chaque étape, la moitié des threads encore "actifs" lit la valeur d'un thread partenaire, effectue l'opération, et met à jour sa propre valeur.

Une observation fine du code révèle que la condition `if (threadIdx.x < offset)` crée un masque de threads actifs contigu (`11...1100...00`). Contrairement à un masque entrelacé (`101010...`), ce pattern est géré très efficacement par les ordonnanceurs des GPU modernes et n'entraîne qu'une pénalité de performance minimale. Le véritable coût de cette approche ne vient donc pas de la divergence, mais de la nécessité d'insérer une barrière de synchronisation de bloc, `__syncthreads()`, après chaque étape de la réduction. Cette instruction est coûteuse car elle force tous les warps d'un bloc à s'attendre mutuellement, créant une latence significative. C'est ce coût de synchronisation, inhérent à toute communication via la mémoire partagée, qui constitue le principal goulot d'étranglement de cette méthode.

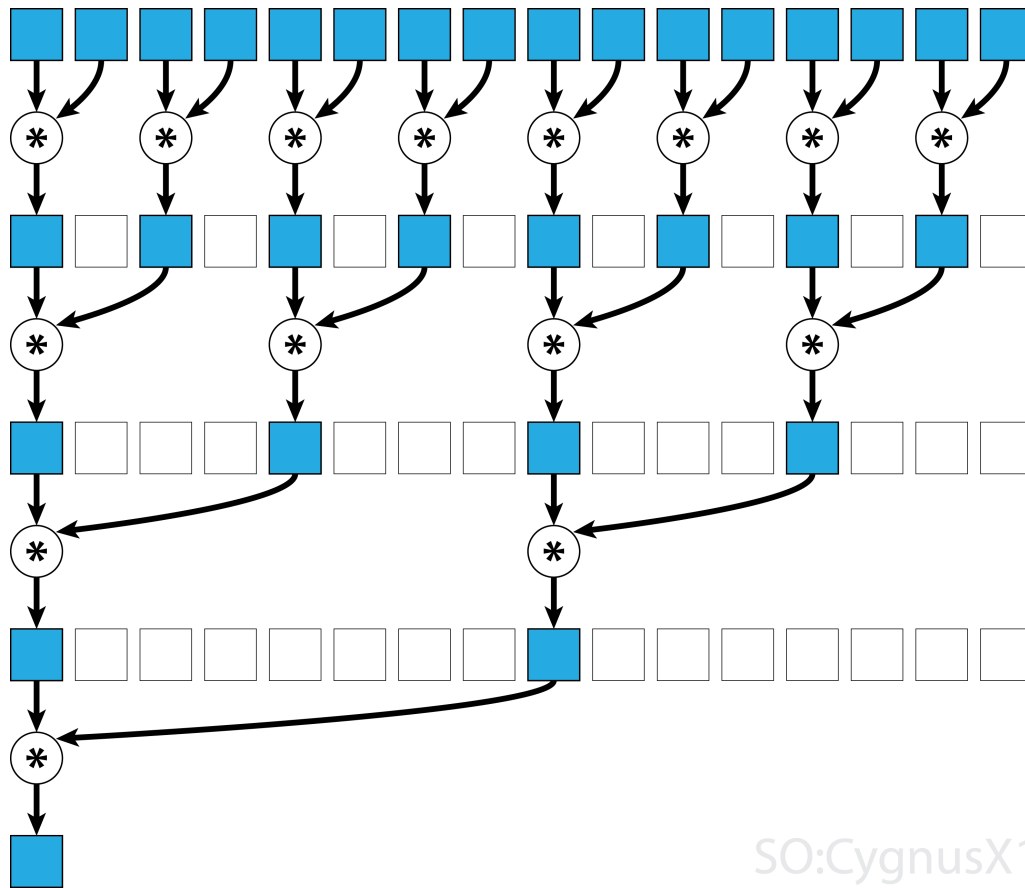


FIGURE 2 – Visualisation de la réduction parallèle en arbre pour 16 threads. Les carrés représentent les données détenues par chaque thread. Le vrai coût de cette méthode n'est pas la divergence (car les threads actifs sont contigus), mais la barrière de synchronisation `__syncthreads()` requise entre chaque niveau de l'arbre.

```

// --- Softmax "online" appliqué sur cette unique tuile de S ---
// 1. Réduction parallèle en MÉMOIRE PARTAGÉE pour trouver le max de chaque ligne
for (int offset = BLOCK_SIZE / 2; offset > 0; offset /= 2) {
    // Cette condition est la source de la divergence de warp
    if (threadIdx.x < offset) {
        sS[threadIdx.y][threadIdx.x] = fmaxf(sS[threadIdx.y][threadIdx.x],
                                              sS[threadIdx.y][threadIdx.x +
                                              ↪ offset]);
    }
    __syncthreads(); // Synchronisation indispensable entre chaque niveau de
    ↪ l'arbre
}
// ... suite du code

```

Listing 2 – Extrait du Kernel V1 montrant la boucle de réduction. La condition `if` est la cause directe de la divergence de warp, une inefficacité qui sera adressée dans les versions ultérieures.

Pour augmenter la vitesse des transferts de données, on adopte le format 16-bits *bfloat16* (Brain Floating Point). Contrairement au format standard *float* sur 32 bits (FP32), le *bfloat16* sacrifie la précision pour préserver la plage dynamique.

TABLE 1 – Comparaison des formats de nombres à virgule flottante.

Format	Bit de Signe	Bits d'Exposant	Bits de Mantisse (Précision)
FP32 (float)	1	8	23
BF16 (bfloat16)	1	8	7
FP16 (half)	1	5	10

Comme le montre le tableau 1, le *bfloat16* conserve les 8 bits d'exposant du FP32, ce qui lui permet de représenter des nombres aussi grands ou aussi petits (même plage dynamique). Cependant, il ne conserve que 7 bits pour la mantisse (contre 23), ce qui réduit drastiquement sa précision. Pour les réseaux de neurones, qui sont intrinsèquement robustes au bruit, ce compromis est extrêmement avantageux : la perte de précision a un impact négligeable sur la convergence du modèle, tandis que le gain de vitesse et la réduction de l'empreinte mémoire sont massifs.

Implémentation du Kernel V1 Le kernel V1 met en œuvre cette fusion en utilisant la mémoire partagée pour stocker temporairement les tuiles de Q , K et de la matrice de score S .

```

__global__ void Kernel_V1_Reference(__nv_bfloat16* Q, __nv_bfloat16* K, ...,
                                   int seq_len, int d_model) {

    // Indices
    int by = blockIdx.y;
    int ty = threadIdx.y, tx = threadIdx.x;
    int row_i = by * BLOCK_SIZE + ty; // Ligne de Q/Out gérée par ce thread
    if (row_i >= seq_len) return;

    // Mémoire partagée pour les tuiles
    __shared__ __nv_bfloat16 sQ[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ __nv_bfloat16 sK[BLOCK_SIZE][BLOCK_SIZE];
    // ... sV, sS etc.

    // Statistiques et accumulateurs en registres
    float m_i = -INFINITY;
    float l_i = 0.0f;
    // ... o_accum

    // Boucle EXTERNE (Flash) sur les blocs de K/V
    for (int j_tile = 0; j_tile < seq_len / BLOCK_SIZE; ++j) {
        float s_accum = 0.0f;

        // Boucle INTERNE (Matmul) pour calculer  $S_{ij} = Q_i * K_j^T$ 
        for (int p = 0; p < d_model / BLOCK_SIZE; p++) {
            sQ[ty][tx] = Q[row_i * d_model + p * BLOCK_SIZE + tx];
            int col_k = j_tile * BLOCK_SIZE + tx;
            sK[tx][ty] = K[col_k * d_model + p * BLOCK_SIZE + ty];
            __syncthreads();

            for (int l = 0; l < BLOCK_SIZE; l++) {
                // On utilise un accumulateur flottant pour limiter la perte de
                // ↪ précision d'un bfloat16*bfloat16
                s_accum += (float)sQ[ty][l] * (float)sK[l][tx];
            }
            __syncthreads();
        }
        // ... Logique du softmax online (max, sum, mise à jour de 0)...
    }
    // ... Écriture finale du résultat ...
}

```

Listing 3 – Structure du Kernel V1, qui fusionne les passes de l’attention. Le code est fonctionnel mais n’est pas optimisé au niveau micro-architectural.

Le point critique de cette version est les lignes de chargement de K et V . Les threads au sein d’un même warp ont des tx contigus (0, 1, 2, ...). Dans l’expression $K[(globCol) * d_model + \dots]$, $globCol$ contient tx , qui est donc multiplié par le grand pas (stride) d_model . Conséquence : des threads adjacents tentent d’accéder à des adresses mémoire très éloignées, ce qui force le matériel à émettre de multiples transactions inefficaces au lieu d’une seule transaction large. C’est le premier goulot d’étranglement majeur que la version V2 va adresser.

4.2.3 Version 2 - Optimisation des Accès par Coalescence

La V1 a posé les bases d’un kernel fonctionnel, mais son analyse a révélé un goulot d’étranglement majeur : les accès mémoire non-coalescés. Cette version V2 s’attaque à ce problème en introduisant le principe fondamental de l’accès **mémoire coalescé**.

Le Problème : L'Accès Mémoire non-coalescé (Strided) Lorsqu'un warp de 32 threads effectue une lecture en mémoire globale, le matériel tente de regrouper ces 32 requêtes en une seule transaction large et efficace. Pour que cela fonctionne, les threads du warp doivent accéder à des adresses mémoire contiguës. Dans le kernel V1, les chargements des matrices K et V violaient ce principe : les threads lisaient des données séparées par une grande foulée (*stride*), comme l'illustre la Figure 3. Le matériel est alors contraint d'émettre de multiples transactions inefficaces, gaspillant la bande passante et introduisant une latence considérable.

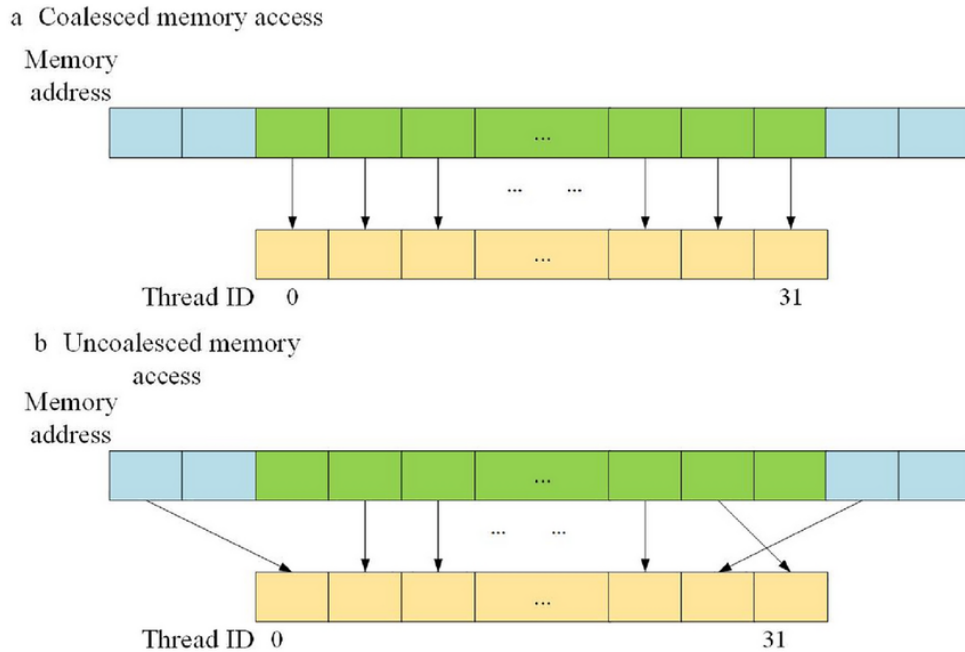


FIGURE 3 – Illustration d'un accès mémoire non-coalescé (strided). Les threads d'un même warp (ex : T0, T1, T2...) accèdent à des adresses mémoires très espacées. Le matériel doit effectuer plusieurs transactions pour récupérer ces données, ce qui est très inefficace.

La Solution : Transposition à la Volée Pour résoudre ce problème, il faut que les threads contigus d'un warp lisent des adresses mémoires contiguës. La solution consiste à lire collaborativement une **ligne** contiguë de la matrice globale. Pour le chargement de K, qui doit être transposé dans la mémoire partagée (sK), on utilise la technique de la **transposition à la volée** : après avoir lu la ligne, chaque thread écrit sa donnée dans une **colonne** de la tuile sK , effectuant la transposition "gratuitement", comme le montre la Figure 4. Pour V, qui ne nécessite pas de transposition, la ligne lue est simplement stockée en tant que ligne dans sV .

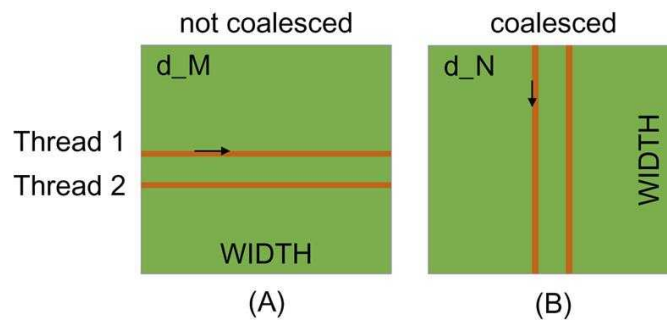


FIGURE 4 – Technique de la transposition à la volée (pour K). Un warp lit une ligne contiguë de la mémoire globale (accès coalescé), puis chaque thread écrit sa valeur dans la colonne correspondante de la mémoire partagée.

Modification du Kernel V2 La mise en œuvre de cette technique ne modifie que les lignes de chargement de K et V pour garantir que tx (l'indice qui varie au sein d'un warp) modifie la partie la moins significative de l'adresse mémoire.

```
__global__ void Kernel_V2_Coalesced(__nv_bfloat16* Q, __nv_bfloat16* K, __nv_bfloat16*
↪ V, ...) {
    // ... indices ...
    int row_i = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    int col_o = blockIdx.x * BLOCK_SIZE + threadIdx.x; // Colonne de la sortie 0

    // Boucle EXTERNE (Flash) sur les blocs de K/V
    for (int j_tile = 0; j_tile < seq_len / BLOCK_SIZE; ++j) {

        // Boucle INTERNE (Matmul) pour S_ij
        for (int p = 0; p < d_model / BLOCK_SIZE; p++) {
            // Chargement de Q (déjà coalescé)
            sQ[ty][tx] = Q[row_i * d_model + p * BLOCK_SIZE + tx];

            // --- DIFF V1 -> V2 (Chargement de K) ---
            // -- V1 (Non-Coalescé) : tx est dans la partie "stride" de l'adresse.
            // sK[tx][ty] = K[(j_tile*BS + tx)*d_model + p*BS + ty];

            // ++ V2 (Coalescé) : tx est dans la partie contiguë.
            sK[tx][ty] = K[(j_tile * BLOCK_SIZE + ty) * d_model + p * BLOCK_SIZE +
↪ tx];

            __syncthreads();
            // ... calcul matmul ...
            __syncthreads();
        }

        // ... calcul du softmax pour obtenir P_ij ...

        // --- DIFF V1 -> V2 (Chargement de V) ---
        // -- V1 (Non-Coalescé) : tx est dans la partie "stride".
        // sV[ty][tx] = V[(j_tile*BS + tx)*d_model + col_o];

        // ++ V2 (Coalescé) : tx est dans la partie contiguë.
        sV[ty][tx] = V[(j_tile * BLOCK_SIZE + ty) * d_model + col_o];
        __syncthreads();

        // ... calcul de la mise à jour de 0 avec P_ij et sV ...
    }
}

```

Listing 4 – Kernel V2 - Correction des chargements de K et V pour garantir des accès coalescés. Cette modification cruciale améliore drastiquement la bande passante mémoire effective.

Avec cette modification, la phase de chargement des données est maintenant optimale. L'analyse de performance révèle alors plus clairement que le goulot d'étranglement s'est déplacé : il s'agit maintenant du coût des multiples barrières de synchronisation `__syncthreads()` requises par la phase de réduction en mémoire partagée.

4.2.4 Version 3 - Optimisation de la Réduction par Primitives Intra-Warp

Le kernel V2 a optimisé le chargement des données, révélant ainsi que le principal goulot d'étranglement restant est la phase de réduction. Le coût ne provient pas de la divergence (minimale), mais des multiples barrières de synchronisation de bloc `__syncthreads()`, qui forcent tous les warps d'un bloc à s'attendre mutuellement. Cette version V3 s'attaque à ce problème en remplaçant la communication via la mémoire partagée par une communication directe entre les registres des threads.

Principe des Primitives "Shuffle" L'architecture CUDA fournit des instructions spéciales, les *shuffles*, qui permettent aux threads d'un même warp d'échanger des données directement depuis leurs registres. La Figure 5 illustre le gain fondamental de cette approche :

- **Sans Shuffle (V2)** : La communication est **indirecte** et se déroule en trois temps : **1** chaque thread écrit sa valeur en mémoire partagée, **2** une coûteuse barrière de synchronisation de bloc `__syncthreads()` est imposée, et enfin **3** le thread "réducteur" (ex : T0) doit relire plusieurs valeurs depuis la mémoire partagée pour effectuer son calcul.
- **Avec Shuffle (V3)** : La communication est **directe** et **atomique**. En une seule instruction `__shfl_down_sync`, le thread T0 reçoit la valeur directement du registre de son partenaire T1. Le flux de données **contourne complètement** la mémoire partagée, éliminant à la fois les accès mémoire et la barrière de synchronisation.

Cela permet de réaliser une réduction en arbre complète au sein de chaque warp avec une latence bien plus faible.

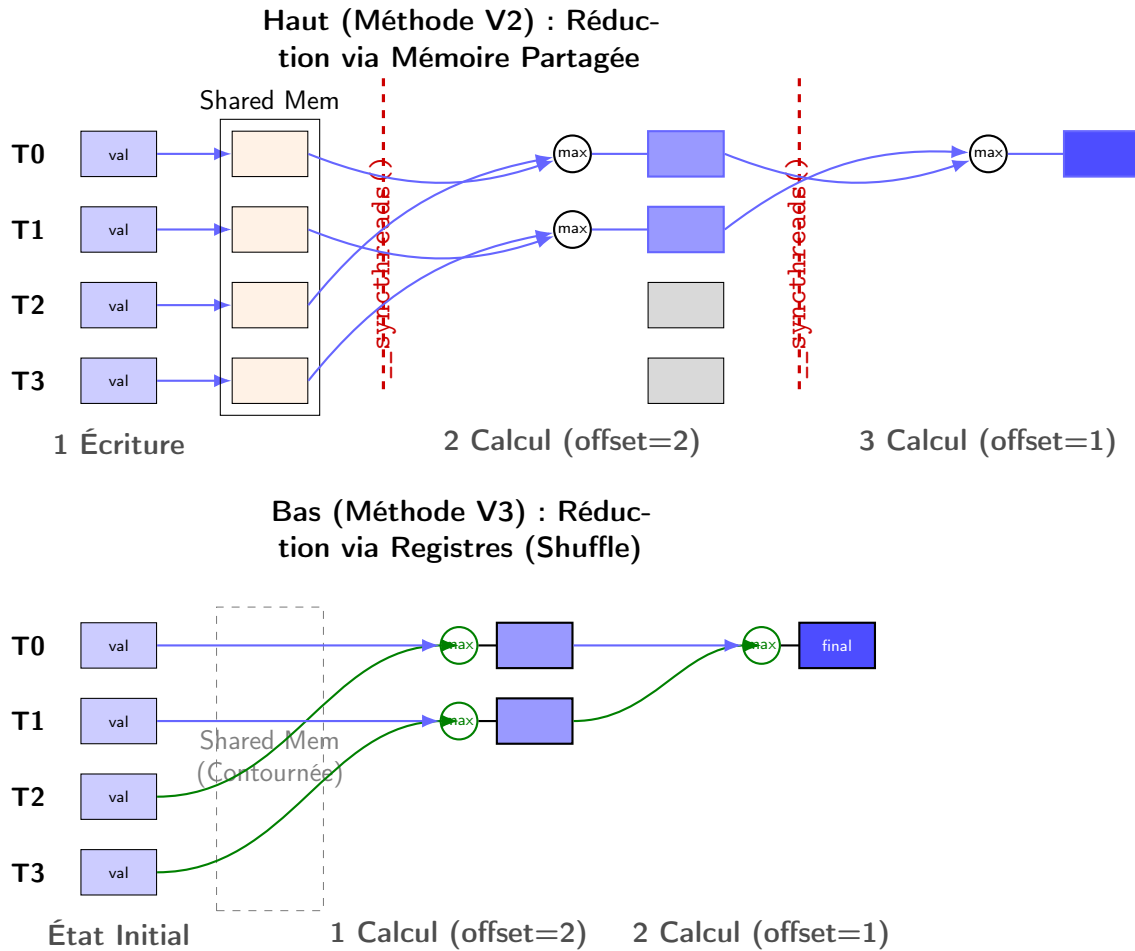


FIGURE 5 – Comparaison des mécanismes de réduction. **Haut (V2)** : un processus lourd en 3 étapes (écriture, synchro, lecture) via la mémoire partagée. **Bas (V3)** : un flux de données direct où les threads actifs reçoivent les valeurs de leurs partenaires de registre à registre, contournant la mémoire partagée et éliminant les synchronisations.

Modification du Kernel V3 La boucle de réduction en mémoire partagée est entièrement remplacée par une séquence d'instructions `shfl_sync`. Chaque ligne de la tuile sS (qui est de taille $BLOCK_SIZE$) est traitée par un ou plusieurs warps. La réduction s'opère donc au niveau du warp.

```

// ... (calcul de la tuile sS comme dans la V2, stocké dans accumulator)
// Le résultat est toujours écrit en mémoire partagée, car il doit être lu
// par d'autres threads pour la multiplication par V. Mais la réduction, elle,
// peut être optimisée.
sS[threadIdx.y][threadIdx.x] = accumulator;
__syncthreads();

// --- DIFF V2 -> V3 ---

// -- V2 (Réduction en Mémoire Partagée)
// for (int offset = BLOCK_SIZE / 2; offset > 0; offset /= 2) {
//     if (threadIdx.x < offset) {
//         sS[threadIdx.y][threadIdx.x] = fmaxf(sS[threadIdx.y][threadIdx.x],
//                                             sS[threadIdx.y][threadIdx.x + offset]);
//     }
//     __syncthreads();
// }
// float row_max = sS[threadIdx.y][0];

// ++ V3 (Réduction en Registres via Shuffle)
// Chaque thread commence avec sa propre valeur d'accumulateur.
float row_max = accumulator;
// On effectue une réduction en arbre au sein de chaque warp.
// La taille du warp est typiquement de 32.
for (int offset = 16; offset > 0; offset /= 2) {
    row_max = fmaxf(row_max, __shfl_down_sync(0xFFFFFFFF, row_max, offset));
}

// À ce stade, le thread 0 de chaque warp détient le max de son warp.
// Si BLOCK_SIZE > 32, une étape finale en mémoire partagée est nécessaire
// pour combiner les résultats des différents warps d'une même ligne.
// (Pour simplifier, on suppose ici BLOCK_SIZE <= 32)
// Le max est ensuite diffusé au sein du warp.
row_max = __shfl_sync(0xFFFFFFFF, row_max, 0);

// Le reste du calcul (exponentielle, somme) est optimisé de la même manière...

```

Listing 5 – Kernel V3 - Remplacement de la réduction en mémoire partagée par une réduction en registres utilisant `__shfl_down_sync`. Les coûteuses barrières `__syncthreads()` de la boucle de réduction sont éliminées.

Cette optimisation élimine le goulot d'étranglement de synchronisation de la phase de réduction, qui devient presque instantanée. La suite logique est de s'atteler aux problèmes de synchronisation lors de la récupération des données.

4.2.5 Version 4 - Masquage de la Latence par Double Buffering (K et V)

Les optimisations précédentes ont amélioré le débit des calculs et des transferts on-chip, mais le kernel reste fondamentalement contraint par la latence des accès à la mémoire globale. Le goulot d'étranglement est le temps d'attente imposé par le chargement des tuiles de K et V . La solution à ce problème réside dans l'implémentation de **deux pipelines imbriqués**, chacune correspondant à une des boucles de l'algorithme.

Principe des Pipelines Imbriqués L'idée est de superposer le chargement des données de l'itération $n + 1$ avec le calcul de l'itération n . Cette technique, appelée *double buffering*, doit être

appliquée aux deux niveaux de l'algorithme :

1. **Un pipeline externe (grossier) sur la boucle j** : Cette boucle itère sur les blocs de V . Le "calcul" à masquer ici est extrêmement long : il s'agit de la totalité du calcul d'une tuile de $S = QK^T$, qui inclut sa propre boucle interne. Ce pipeline va donc lancer le chargement de la tuile V_{j+1} en arrière-plan pendant que tout le travail lié à V_j est effectué.
2. **Un pipeline interne (fin) sur la boucle p** : Cette boucle itère sur les blocs de K pour calculer la multiplication matricielle. Le "calcul" à masquer ici est plus court : il s'agit de la multiplication d'une sous-tuile. Ce pipeline va superposer le chargement de K_{p+1} avec le calcul utilisant K_p .

La pipeline interne sur K est donc entièrement contenu dans la phase de "calcul" de la pipeline externe sur V . C'est cette hiérarchie qui permet de masquer efficacement les deux sources principales de latence mémoire. La Figure 6 illustre le principe pour une seule de ces pipelines ; la stratégie complète consiste à imbriquer ce mécanisme.

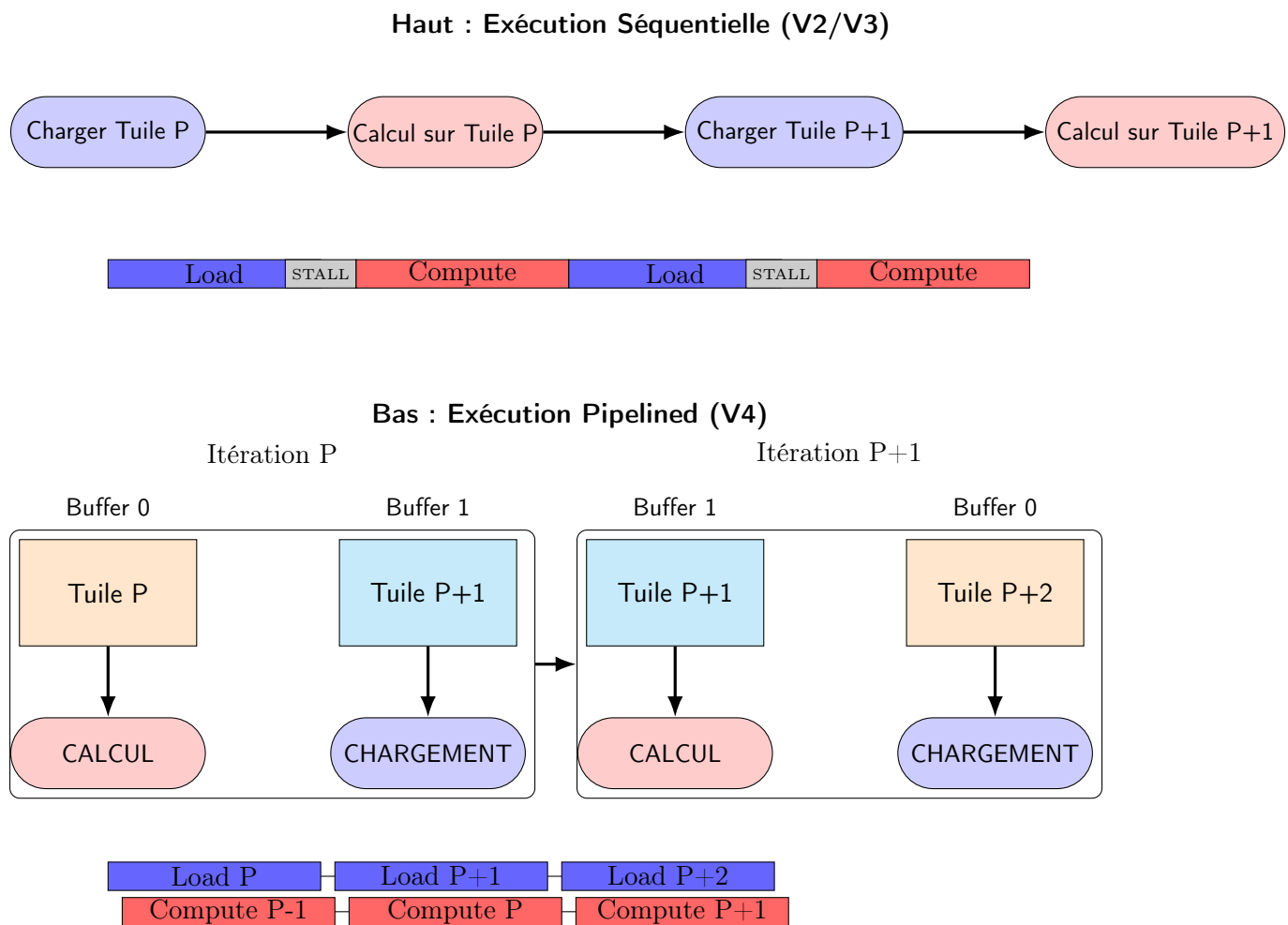


FIGURE 6 – Principe du double buffering. Le diagramme illustre le concept de superposition pour un pipeline (e.g. celui sur K). Le chargement de la tuile $P+1$ est superposé avec le calcul sur la tuile P . L'implémentation V4 complète imbrique deux pipelines de ce type.

Modification du Kernel V4 L'implémentation requiert de doubler la taille de la mémoire partagée pour K et V , et de restructurer les deux boucles pour suivre le pattern PROLOGUE -> BOUCLE -> EPILOGUE.

```

// Mémoire partagée doublée pour les deux pipelines
__shared__ __nv_bfloat16 sK[2][BLOCK_SIZE][BLOCK_SIZE];
__shared__ __nv_bfloat16 sV[2][BLOCK_SIZE][BLOCK_SIZE];

// --- PROLOGUE DU PIPELINE EXTERNE (V) ---
// Charge la première tuile V_0.
// load_V_tile(V, 0, sV[0]);

int v_buf = 0;
// --- BOUCLE EXTERNE 'j' (PIPELINE SUR V) ---
for (int j = 0; j < seq_len / BLOCK_SIZE; ++j) {
    __syncthreads(); // SYNC : Garantit que V_j (chargé à j-1) est prêt.

    // PENDANT que le calcul sur V_j se déroule, on lance le chargement de V_{j+1}
    int next_v_buf = 1 - v_buf;
    if (j < seq_len / BLOCK_SIZE - 1) {
        // LOAD : Lancement asynchrone du chargement de la PROCHAINE tuile V
        // load_V_tile(V, j + 1, sV[next_v_buf]);
    }

    // COMPUTE : Phase de calcul pour l'itération 'j', utilisant sV[v_buf].
    // Cette phase contient le pipeline interne complet sur K.
    {
        // --- PROLOGUE DU PIPELINE INTERNE (K) ---
        // load_K_tile(K, j, 0, sK[0]);
        int k_buf = 0;

        // --- BOUCLE INTERNE 'p' (PIPELINE SUR K) ---
        for (int p = 0; p < d_model / BLOCK_SIZE; ++p) {
            __syncthreads(); // SYNC : Garantit que K_p est prêt.
            int next_k_buf = 1 - k_buf;
            if (p < d_model / BLOCK_SIZE - 1) {
                // LOAD : Lancement asynchrone de K_{p+1}
                // load_K_tile(K, j, p + 1, sK[next_k_buf]);
            }

            // COMPUTE : Calcul avec K_p (qui est dans sK[k_buf])
            // mma_sync( ... sQ, sK[k_buf] ... );
            k_buf = next_k_buf;
        }
        // ... Épilogue du pipeline K ...
    }
    // ... fin de la phase compute du pipeline V ...

    // Après le calcul de  $QK^T$ , utiliser V_j, qui est dans sV[v_buf].
    // mma_sync(P_j, sV[v_buf], 0_accum);

    v_buf = next_v_buf;
}
// --- EPILOGUE DU PIPELINE EXTERNE (V) ---
// (aucun si le dernier calcul est dans la boucle)

```

Listing 6 – Kernel V4 - Illustration de l’architecture à deux pipelines imbriqués. La boucle externe j gère le pipeline de V , tandis que la boucle interne p gère celui de K .

Cette optimisation est la dernière optimisation de masquage de latence que la durée du stage

a permis d'implémenter et de tester. Une piste d'exploration subséquente serait la spécialisation de warp : L'idée que N warps s'occupent des calculs, et $total - N$ s'occupent des chargements.

4.2.6 Version 5 - Accélération Matérielle via les Tensor Cores (WMMA)

Le kernel V4 a atteint les limites des optimisations de flux de données explorées durant ce projet. Le goulot d'étranglement est désormais revenu au calcul lui-même, en particulier les deux multiplications matricielles ($S = QK^T$ et $O = PV$). Cette version V5 franchit la dernière étape de l'optimisation en déléguant ces calculs aux unités matérielles spécialisées du GPU : les **Tensor Cores**.

Le Dernier Maillon : les Unités de Calcul Spécialisées Les GPU NVIDIA modernes intègrent des circuits dédiés à l'accélération des opérations de type "produit matriciel et accumulation" (Matrix Multiply-Accumulate, MMA). Pour les exploiter, on utilise l'API C++ **WMMA (Warp-Matrix Multiply-Accumulate)** [8], qui abstrait leur programmation derrière le concept de **fragment** : une poignée représentant un ensemble de registres organisés pour contenir une sous-matrice.

L'utilisation des Tensor Cores suit un cycle strict en quatre étapes, comme l'illustre la Figure 7. Ce cycle est appliqué deux fois dans notre algorithme : une première fois pour $S = QK^T$, et une seconde fois pour $O = PV$.

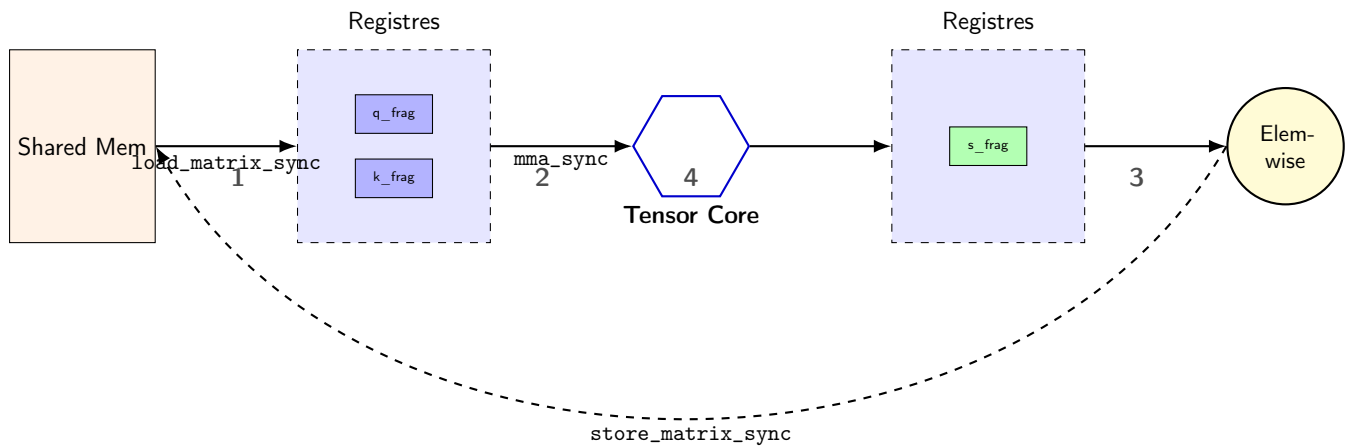


FIGURE 7 – Le cycle de calcul WMMA. **1. LOAD** : les données passent de la mémoire partagée aux registres. **2. MMA** : l'instruction est exécutée par le Tensor Core. **3. MANIPULATION IN-REGISTER** : des opérations (ex : scaling) sont appliquées directement sur le fragment accumulateur. **4. STORE** : le résultat est déchargé vers la mémoire partagée pour les étapes suivantes.

- 1. LOAD** : Chargement des données (sQ , sK , ou plus tard sP , sV) de la mémoire partagée vers les fragments en registres via `wmma::load_matrix_sync`.
- 2. MMA** : Exécution de la multiplication via `wmma::mma_sync`. Le résultat est accumulé dans un fragment, toujours dans les registres.
- 3. MANIPULATION IN-REGISTER** : Avant de décharger le résultat, des opérations *element-wise* (comme la mise à l'échelle par $\frac{1}{\sqrt{d_k}}$) sont appliquées directement sur les éléments du fragment accumulateur. Cette étape est cruciale car elle évite un aller-retour coûteux en mémoire partagée.

4. **STORE** : Pour que les données du résultat soient accessibles aux opérations qui ne peuvent pas opérer sur des fragments (comme la réduction du softmax), elles doivent être déchargées des registres vers la mémoire partagée avec `wmma::store_matrix_sync`.

C'est cette dernière étape, le *STORE*, qui devient le nouveau point de contention. Le calcul MMA est si rapide que le temps passé à transférer le résultat des registres vers la mémoire partagée n'est plus négligeable.

Modification du Kernel V5 Les boucles de multiplication manuelles des deux phases de calcul sont remplacées par le cycle WMMA.

```

// ... Dans la boucle interne 'p' ...

// Déclaration des fragments pour  $S = QK^T$ 
wmma::fragment<...> q_frag, k_frag;
wmma::fragment<wmma::accumulator, ...> s_accum_frag;
wmma::fill_fragment(s_accum_frag, 0.0f);

// --- Phase 1 : Calcul de  $S = QK^T$  ---
// Boucle 'p' (pipeline K)
for (int p = 0; ...) {
    // ... synchro et chargement de K dans sK[current_k_buf] ...

    // 1. LOAD:
    wmma::load_matrix_sync(q_frag, &sQ[...] [...], ...);
    wmma::load_matrix_sync(k_frag, &sK[current_k_buf][0][0], ...);

    // 2. MMA:
    wmma::mma_sync(s_accum_frag, q_frag, k_frag, s_accum_frag); //L'accumulateur est
    ↪ un produit de bfloat16, il devient donc un flottant. Le résultat étant stocké
    ↪ très vite, ce gain de précision n'est pas réellement exploité.
}

// 3. MANIPULATION IN-REGISTER:
float scale = 1.0f / sqrtf((float)d_model);
for(int i=0; i < s_accum_frag.num_elements; ++i) {
    s_accum_frag.x[i] *= scale;
}

// 4. STORE: Déchargement vers la mémoire partagée. C'est le NOUVEAU GOULOT.
__shared__ float sS[BLOCK_SIZE][BLOCK_SIZE];
wmma::store_matrix_sync(&sS[0][0], s_accum_frag, ...);
__syncthreads();

// ... Le code du softmax (réduction, etc.) opère sur sS ...
// ... Il calcule la matrice P et la stocke dans une sP en mémoire partagée ...

// --- Phase 2 : Calcul de  $O = PV$  ---
// Déclaration des fragments pour  $O = PV$ 
wmma::fragment<...> p_frag, v_frag;
wmma::fragment<wmma::accumulator, ...> o_accum_frag;
// ...
// 1. LOAD: Charger sP et sV[current_v_buf]
wmma::load_matrix_sync(p_frag, &sP[...] [...], ...);
wmma::load_matrix_sync(v_frag, &sV[current_v_buf][0][0], ...);
// 2. MMA:
wmma::mma_sync(o_accum_frag, p_frag, v_frag, o_accum_frag);
// ...

```

Listing 7 – Kernel V5 - Remplacement des deux multiplications matricielles par le cycle WMMA. Cette accélération massive expose le coût du `store_matrix_sync` comme le nouveau goulot d'étranglement.

Cette version V5 représente l'état de l'art des optimisations conventionnelles. Le calcul est maintenant massivement accéléré, mais cette accélération même met en lumière un nouveau goulot d'étranglement, plus subtil : le cycle **Registre** → **Mémoire Partagée** → **Registre** nécessaire pour enchaîner le calcul MMA avec les opérations de réduction du softmax. C'est la

résolution de ce dernier obstacle qui motivera la phase finale de ce projet.

4.2.7 Résultats

L'analyse des résultats présentés dans le tableau 2 confirme l'intérêt de la démarche d'optimisation incrémentale. Chaque version du kernel apporte une réponse ciblée à un goulot d'étranglement spécifique, et les gains de performance mesurés permettent de quantifier l'impact de chaque décision architecturale.

Le premier constat est le saut de performance spectaculaire entre la version naïve V0 et la version V1 (511 ms $\rightarrow$ 0.63 ms), soit une accélération de près de $800\times$. Ce gain spectaculaire n'est pas une simple optimisation, mais le résultat d'un changement algorithmique *fondamental* : la fusion des passes de calcul. En éliminant les lectures redondantes depuis la mémoire globale, cette approche de type **FlashAttention** établit une baseline moderne et performante, sur laquelle les optimisations micro-architecturales ultérieures pourront s'appuyer.

À partir de cette base solide, la trajectoire des versions V1 à V5 illustre une chasse aux goulots d'étranglement de plus en plus *subtils*. Le passage à V2 (+10%) a démontré l'importance critique des accès coalescés pour saturer la bande passante mémoire. V3 (+15%) a prouvé que la synchronisation inter-warp via la mémoire partagée était un coût non négligeable, résolu par l'utilisation des primitives **shuffle**. V4 (+4%) a ensuite réussi à masquer une partie de la latence résiduelle des accès mémoire grâce au **double buffering**. Enfin, le passage à V5 (+30%) a marqué le dernier grand bond en avant, en déléguant le calcul matriciel aux Tensor Cores via l'API **WMMA**. Cette dernière étape a réduit la latence à un plancher de 0.33 ms, représentant l'état de l'art des techniques d'optimisation conventionnelles explorées.

Cependant, l'analyse de cette version V5 avec **Nsight Compute** a constitué le véritable *point de bascule* du projet. Alors que la performance brute était excellente, le profileur a révélé que le calcul MMA était devenu si rapide qu'il exposait un nouveau goulot, presque invisible auparavant : le coût du cycle **Registre** $\rightarrow$ **Mémoire Partagée** $\rightarrow$ **Registre** nécessaire pour décharger les résultats du calcul matriciel afin d'effectuer la réduction du softmax. Le plafond de performance n'était plus dicté par l'accès à la mémoire globale ou la puissance de calcul, mais par l'efficacité des mouvements de données *on-chip*. C'est cette conclusion qui a motivé le pivot vers la phase finale et la plus exploratoire du projet : le développement d'une méthode radicale pour supprimer entièrement ce transit.

4.2.8 Le Pivot : Synthèse des Résultats et Identification du Goulot Résiduel

L'analyse des performances de cette suite de kernels a mis en évidence un plafond de performance. Même avec toutes les optimisations standards combinées (tiling, coalescing, double buffering, WMMA), l'analyse micro-architecturale avec **Nsight Compute** a révélé la persistance d'un cycle coûteux de transfert vers la mémoire partagée.

4.3 Reverse-Engineering du Layout WMMA

L'API **WMMA** de CUDA abstrait l'utilisation des Tensor Cores derrière le concept de *fragment*, une structure de données opaque qui représente une sous-matrice directement dans les registres d'un warp. Si cette abstraction simplifie le développement, elle agit comme une "boîte noire" qui interdit toute manipulation directe des données une fois chargées. Pour dépasser cette limitation et permettre des opérations *in-register*, une première étape indispensable était de cartographier la structure interne de cette boîte noire par reverse-engineering.

Comme cette organisation des données n'est pas documentée par NVIDIA, un micro-kernel a été spécifiquement développé pour la sonder expérimentalement. Le principe, décrit en détail dans l'Algorithme 1, repose sur l'utilisation d'une "matrice traceur". Une matrice de 16x16 est initialisée en mémoire partagée, où la valeur de chaque élément est une construction numérique unique qui encode ses propres coordonnées (par exemple, `valeur = ligne * 100 + colonne`). Cette matrice est ensuite chargée dans un fragment WMMA et passée à travers une multiplication par la matrice identité, une opération qui a pour effet de préserver les valeurs de la matrice traceur tout en les réorganisant selon le layout interne du fragment.

Le fragment résultant, qui contient maintenant la carte de sa propre structure, est finalement déchargé en mémoire globale. Un script sur l'hôte peut alors lire ce buffer et reconstruire la table de correspondance exacte entre une coordonnée logique (ligne, colonne) et sa position physique réelle : le thread (*lane*) qui la stocke et l'index de son registre (*frag_idx*).

Algorithm 1 WMMA Fragment Layout Reverse-Engineering

```

1: Input : None
2: Output : A 16x16 table Mapping[row][col] = (lane, frag_idx)
3: procedure DEVICEKERNEL(output_buffer)
4:   tracer_matrix ← 16x16 shared memory array
5:   lane ← thread's lane index (0-31)
6:   // Each thread in the block populates one element of the tracer matrix
7:   tracer_matrix[threadIdx.y][threadIdx.x] ← threadIdx.y * 100 + threadIdx.x
8:   __syncthreads()
9:   if lane < 32 then                                ▷ Code executed by a single warp
10:    a_frag ← wmma::load_matrix_sync(tracer_matrix)
11:    id_frag ← Load Identity Matrix into a fragment
12:    acc_frag ← wmma::mma_sync(a_frag, id_frag)          ▷ Fragment now holds tracer
values
13:    for i ← 0 to 7 do                                ▷ Dump fragment registers to global memory
14:      output_buffer[lane * 8 + i] ← acc_frag.x[i]
15:    end for
16:  end if
17: end procedure
18: procedure HOSTANALYSIS(output_buffer)
19:   for lane ← 0 to 31 do
20:     for frag_idx ← 0 to 7 do
21:       value ← output_buffer[lane * 8 + frag_idx]
22:       row ← ⌊value/100⌋
23:       col ← value%100
24:       Mapping[row][col] ← (lane, frag_idx)
25:     end for
26:   end for
27:   return Mapping
28: end procedure

```

L'exécution de cette procédure révèle une structure de données entrelacée (ou *swizzled*), mais qui, de manière cruciale, est parfaitement régulière et prédictible. La Figure 8 présente le mapping complet obtenu sur une architecture NVIDIA RTX 5080.

FIGURE 8 – Layout de données du fragment WMMA (16x16) découvert expérimentalement sur une RTX 5080. Chaque cellule indique le *lane* (ID de thread de 0 à 31) qui stocke l'élément de matrice correspondant. On observe un motif complexe mais parfaitement répétitif toutes les 8 lignes et 8 colonnes, formant des "quadrants" de mapping identiques. Par exemple, l'élément en (R00, C02) est détenu par le thread 1.

	C00	C01	C02	C03	C04	C05	C06	C07	C08	C09	C10	C11	C12	C13	C14	C15
R00	0	0	1	1	2	2	3	3	0	0	1	1	2	2	3	3
R01	4	4	5	5	6	6	7	7	4	4	5	5	6	6	7	7
R02	8	8	9	9	10	10	11	11	8	8	9	9	10	10	11	11
R03	12	12	13	13	14	14	15	15	12	12	13	13	14	14	15	15
R04	16	16	17	17	18	18	19	19	16	16	17	17	18	18	19	19
R05	20	20	21	21	22	22	23	23	20	20	21	21	22	22	23	23
R06	24	24	25	25	26	26	27	27	24	24	25	25	26	26	27	27
R07	28	28	29	29	30	30	31	31	28	28	29	29	30	30	31	31
R08	0	0	1	1	2	2	3	3	0	0	1	1	2	2	3	3
R09	4	4	5	5	6	6	7	7	4	4	5	5	6	6	7	7
R10	8	8	9	9	10	10	11	11	8	8	9	9	10	10	11	11
R11	12	12	13	13	14	14	15	15	12	12	13	13	14	14	15	15
R12	16	16	17	17	18	18	19	19	16	16	17	17	18	18	19	19
R13	20	20	21	21	22	22	23	23	20	20	21	21	22	22	23	23
R14	24	24	25	25	26	26	27	27	24	24	25	25	26	26	27	27
R15	28	28	29	29	30	30	31	31	28	28	29	29	30	30	31	31

4.4 Stratégie de Réorganisation des Données : La Permutation Partielle

La connaissance du layout des données est la clé pour concevoir un algorithme de manipulation efficace. Une analyse plus approfondie de la Figure 8 révèle que les éléments d'une même ligne de la matrice sont systématiquement répartis entre 8 threads différents, et au sein de chaque thread, ces éléments sont dispersés dans des registres non-contigus.

Une approche naïve consisterait à "détricoter" entièrement cette structure pour restaurer un layout linéaire en mémoire. Cependant, une telle opération serait prohibitivement coûteuse, nécessitant de multiples instructions de communication inter-thread (`__shfl_sync`) pour chaque élément. L'objectif n'est pas la linéarisation, mais la **consolidation** : regrouper les éléments d'une même ligne pour permettre une réduction parallèle.

La solution développée est une **permutation partielle in-register**. Comme l'illustre conceptuellement la Figure 9, une unique opération de SWAP entre des paires de registres spécifiques au sein de chaque thread suffit à atteindre cet objectif. Cette opération, qui ne coûte que quelques cycles d'horloge, a pour effet de regrouper tous les éléments appartenant à une même ligne logique dans des registres physiquement contigus au sein de sous-groupes de 4 threads (les "quads").

FIGURE 9 – Vue conceptuelle de la permutation in-register pour un seul thread (ici, le *lane* 0). L'opération de SWAP intervertit les registres contenant des segments des lignes 0 et 8. Après la permutation, les quatre registres contenant les données de la Ligne 0 sont contigus, tout comme ceux de la Ligne 8, ce qui les prépare à une réduction locale efficace.

LANE 0 REGISTERS: BEFORE vs AFTER PERMUTATION

BEFORE (Data for a single row is scattered)		AFTER (Data is consolidated)	
x[0,1]	ROW 0	x[0,1]	ROW 0
x[2,3]	ROW 8	x[2,3]	ROW 0
x[4,5]	ROW 0	x[4,5]	ROW 8
x[6,7]	ROW 8	x[6,7]	ROW 8

Ce mapping, bien que complexe, se révèle être entièrement déterministe et peut être décrit par un ensemble de formules mathématiques. Ces formules, dérivées de l'analyse du mapping, constituent la "loi" qui gouverne la structure des données. Pour une coordonnée (ligne, colonne) dans la matrice, sa position physique (*lane*, *i*) est donnée par :

$$\text{row} = (\text{lane}/4) + (((i \gg 1) \& 1) \times 8) \quad (2)$$

$$\text{col} = ((\text{lane} \% 4) \times 2) + (i \% 2) + ((i \gg 2) \times 8) \quad (3)$$

La maîtrise de ces formules est doublement cruciale. Non seulement elles justifient la stratégie de permutation, mais elles sont aussi indispensables pour l'opération inverse : écrire un résultat, stocké de manière contiguë dans les registres d'un thread, à sa position correcte et entrelacée dans la mémoire globale. L'Algorithme 2 détaille cette procédure d'écriture.

Algorithm 2 Stockage de Données Locales via Mapping Direct

```

1: Input : Un tableau local local_data[0..7], une matrice de sortie globale Out
2: Context : Exécuté par chaque thread lane dans un warp
3: block_row_start  $\leftarrow$  Index de ligne de départ de la tuile de sortie
4: block_col_start  $\leftarrow$  Index de colonne de départ de la tuile de sortie
5: for i  $\leftarrow$  0 to 7 do
6:   // Calcul des coordonnées cibles via les formules de mapping direct
7:   target_row  $\leftarrow$  block_row_start + (lane/4) + (((i  $\gg$  1)  $\&$  1)  $\times$  8)
8:   target_col  $\leftarrow$  block_col_start + ((lane  $\%$  4)  $\times$  2) + (i  $\%$  2) + ((i  $\gg$  2)  $\times$  8)
9:   // Écriture de la donnée locale à sa position correcte dans la sortie globale
10:  Out[target_row][target_col]  $\leftarrow$  local_data[i]
11: end for

```

4.5 Exécution : Réduction Parallèle In-Register

Une fois les données logiquement alignées par la permutation, le terrain est prêt pour une réduction massivement parallèle. L'analyse du mapping montre que les 16 lignes de la matrice

sont gérées par 8 sous-groupes de 4 threads ("quads"). Chaque quad est responsable de deux lignes distinctes. La réduction s'opère donc en deux étapes, décrites dans l'Algorithme 3.

La première étape est une **réduction locale**, effectuée indépendamment par chaque thread sur les 4 registres contigus qui contiennent désormais les données d'une ligne. La seconde est une **réduction parallèle intra-quad**, où les 4 threads d'un même sous-groupe collaborent pour agréger leurs résultats locaux.

La clé de cette seconde étape est la construction d'un masque de bits dynamique pour l'instruction `__shfl_xor_sync`. Le masque `0xF` $\ll ((lane / 4) * 4)$ n'est pas une valeur arbitraire ; il est le résultat direct de la structure du mapping. Il crée une "fenêtre" de communication de 4 bits qui isole chaque quad (par exemple, `0x0...0F` pour les threads 0-3, `0x0...F0` pour les threads 4-7, etc.). Cette technique permet aux 8 quads du warp d'effectuer leur réduction simultanément, en parallèle, sans aucune interférence, atteignant ainsi un parallélisme de données maximal.

Algorithm 3 Réduction Parallèle Générique In-Register

```

1: Input : Un fragment accumulateur permuté acc_frag, un opérateur binaire associatif  $\oplus$ 
2: Output : Pour chaque quad, deux valeurs réduites val_A et val_B
3: Context : Exécuté par chaque thread lane dans un warp
4:
5: // Étape 1 : Réduction locale au sein des registres de chaque thread
6: local_A  $\leftarrow$  identité
7: for k  $\leftarrow$  0 to 3 do
8:   local_A  $\leftarrow$  local_A  $\oplus$  acc_frag.x[k]
9: end for
10: local_B  $\leftarrow$  identité
11: for k  $\leftarrow$  4 to 7 do
12:   local_B  $\leftarrow$  local_B  $\oplus$  acc_frag.x[k]
13: end for
14:
15: // Étape 2 : Réduction parallèle au sein de chaque quad de 4 threads
16: mask  $\leftarrow$  0xF  $\ll ((lane/4) * 4)$  ▷ e.g., 0xF pour le quad 0, 0xF0 pour le quad 1
17: partner_A  $\leftarrow$  __shfl_xor_sync(mask, local_A, 1)
18: val_A  $\leftarrow$  local_A  $\oplus$  partner_A
19: partner_A  $\leftarrow$  __shfl_xor_sync(mask, val_A, 2)
20: val_A  $\leftarrow$  val_A  $\oplus$  partner_A
21:
22: // Répétition pour le second ensemble de données
23: partner_B  $\leftarrow$  __shfl_xor_sync(mask, local_B, 1)
24: val_B  $\leftarrow$  local_B  $\oplus$  partner_B
25: partner_B  $\leftarrow$  __shfl_xor_sync(mask, val_B, 2)
26: val_B  $\leftarrow$  val_B  $\oplus$  partner_B
27:
28:
29: if lane%4 == 0 then ▷ Le thread leader du quad détient maintenant les valeurs finales
30:   return val_A, val_B
31: end if

```

4.6 Intégration dans un Cas d'Usage : Le Softmax de FlashAttention

Le véritable potentiel de ces briques élémentaires (reverse-engineering, permutation, réduction) se réalise lorsqu'elles sont assemblées pour remplacer une séquence complexe et coûteuse au sein d'un kernel de pointe. Dans le cadre de FlashAttention, la phase de calcul du softmax sur une tuile S_{ij} implique typiquement de multiples allers-retours en mémoire partagée.

Notre technique permet de remplacer entièrement cette séquence par un flux d'opérations se déroulant exclusivement dans les registres. L'Algorithme 4 détaille cet enchaînement. Après le calcul de S_{ij} , le fragment est mis à l'échelle, permuté, puis les statistiques (max, somme) sont calculées via la réduction in-register. Les statistiques globales sont mises à jour, le fragment est normalisé pour obtenir la matrice de probabilités P_{ij} , puis la permutation inverse est appliquée pour préparer le fragment à la multiplication suivante ($P_{ij}V_j$). Chaque étape, y compris la mise à jour des statistiques "online", s'exécute sans aucun accès à la mémoire partagée, éliminant ainsi les latences et les barrières de synchronisation associées.

Algorithm 4 Séquence Softmax Fissionnée et In-Register pour une Tuile FlashAttention

```

1: Input : Fragment accumulateur work_frag (contenant  $S_{ij}$ ), stats précédentes  $m_{old}, l_{old}$ 
2: Output : Fragment de probabilité p_frag (contenant  $P_{ij}$ ), stats mises à jour  $m_{new}, l_{new}$ 
3: Context : Exécuté par chaque thread lane dans un warp après le mma_sync de  $Q_i K_j^T$ 
4:
5: // 1. Mise à l'échelle et Permutation
6: work_frag  $\leftarrow$  work_frag /  $\sqrt{d_k}$ 
7: Appliquer la permutation in-register sur work_frag (selon la Figure 9)
8:
9: // 2. Calcul du max de la tuile courante ( $m_{ij}$ )
10:  $m_{local\_A}, m_{local\_B} \leftarrow$  Effectuer la réduction Max in-register (Algorithme 3) sur work_frag
11:
12: // 3. Mise à jour du max global
13:  $m_{new} \leftarrow \max(m_{old}, m_{ij})$ 
14:  $l_{old\_rescaled} \leftarrow l_{old} \times \exp(m_{old} - m_{new})$ 
15:
16: // 4. Calcul de la somme de la tuile courante ( $l_{ij}$ )
17: work_frag  $\leftarrow \exp(\text{work\_frag} - m_{new})$ 
18:  $l_{local\_A}, l_{local\_B} \leftarrow$  Effectuer la réduction Somme in-register (Algorithme 3) sur work_frag
19:
20: // 5. Mise à jour du normalisateur global
21:  $l_{new} \leftarrow l_{old\_rescaled} + l_{ij}$ 
22:
23: // 6. Normalisation finale et préparation pour la suite
24: p_frag  $\leftarrow$  work_frag /  $l_{new}$ 
25: Appliquer la permutation inverse sur p_frag pour préparer la multiplication  $P_{ij}V_j$ .
```

5 Résultats Obtenus

Cette section présente l'ensemble des mesures de performance et d'efficacité énergétique collectées au cours du projet. L'analyse est structurée en deux temps : d'abord, l'évaluation de la suite d'optimisations incrémentales qui ont mené à un kernel baseline hautement optimisé ; ensuite, la validation de la technique in-register, en la confrontant à cette baseline sur des cas d'usage isolés et de bout-en-bout.

5.1 Protocole Expérimental

Pour garantir la rigueur et la reproductibilité des mesures, un protocole de benchmark strict et automatisé a été mis en œuvre.

- **Mesure de Latence** : La performance est mesurée en utilisant l'API CUDA Events pour une précision maximale. Chaque mesure est la moyenne de 1000 exécutions du kernel,

précédées par 100 exécutions de chauffe ("warm-up") pour stabiliser la fréquence du GPU et l'état des caches.

- **Mesure Énergétique** : La consommation est mesurée en soumettant le GPU à une charge de travail soutenue (le kernel est lancé en boucle continue). L'outil `nvidia-smi` est utilisé pour échantillonner la puissance consommée (*power draw*) à haute fréquence sur une période de 10 secondes. La valeur moyenne est retenue comme la consommation de référence du kernel.
- **Validation de l'Exactitude** : Avant tout benchmark, l'exactitude de chaque kernel a été validée en comparant ses résultats à une implémentation de référence en FP32 sur CPU, avec une marge d'erreur numérique acceptable pour le format `bfloat16`.

L'ensemble de ce protocole est orchestré par une suite de scripts et un `Makefile`, assurant une exécution identique des tests pour chaque version de kernel.

5.2 Analyse de la Démarche d'Optimisation Incrémentale

Le tableau 2 synthétise l'évolution des métriques de performance à travers la suite de kernels développés. Chaque ligne représente une étape d'optimisation, répondant au goulot d'étranglement identifié dans la version précédente.

TABLE 2 – Évolution des métriques de performance et d'efficacité sur une NVIDIA RTX 5080 (seq_len=4096, d_model=64). Chaque version adresse le goulot d'étranglement identifié dans la précédente.

Version	Optimisation Principale	Latence (ms)	Nouveau Goulot Révélé
V0 - Naïve	Algorithme littéral	511.61	Accès multiples et redondants à la HBM
V1 - Fusion	Fusion de kernel (type FA-1)	0.632	Accès mémoire non-coalescés (strided)
V2 - Coalesced	Coalescence des accès HBM	0.574	Synchronisation inter-warp (Shared Mem)
V3 - Shuffle	Réduction via primitives intra-warp	0.497	Latence des chargements depuis la HBM
V4 - DB	Double Buffering (Pipeline)	0.478	Vitesse du calcul arithmétique
V5 - WMMA	Accélération via Tensor Cores	0.338	Store Registre vers Shared Mem

L'analyse de ces résultats confirme la validité de la démarche. Le passage de V0 à V1 (511 ms $\rightarrow$ 0.63 ms), une accélération de près de 800x, illustre le changement de paradigme imposé par les algorithmes *I/O-aware* comme FlashAttention. Par la suite, chaque optimisation micro-architecturale apporte un gain mesurable et expose un goulot d'étranglement plus subtil : la coalescence (V2, +10%), l'élimination des synchronisations de la réduction (V3, +15%), le masquage de latence (V4, +4%) et enfin, l'utilisation du matériel spécialisé (V5, +30%). Cette trajectoire culmine avec le kernel V5, une baseline hautement optimisée qui expose le dernier obstacle à la performance on-chip : le coût du transit des données entre les registres et la mémoire partagée.

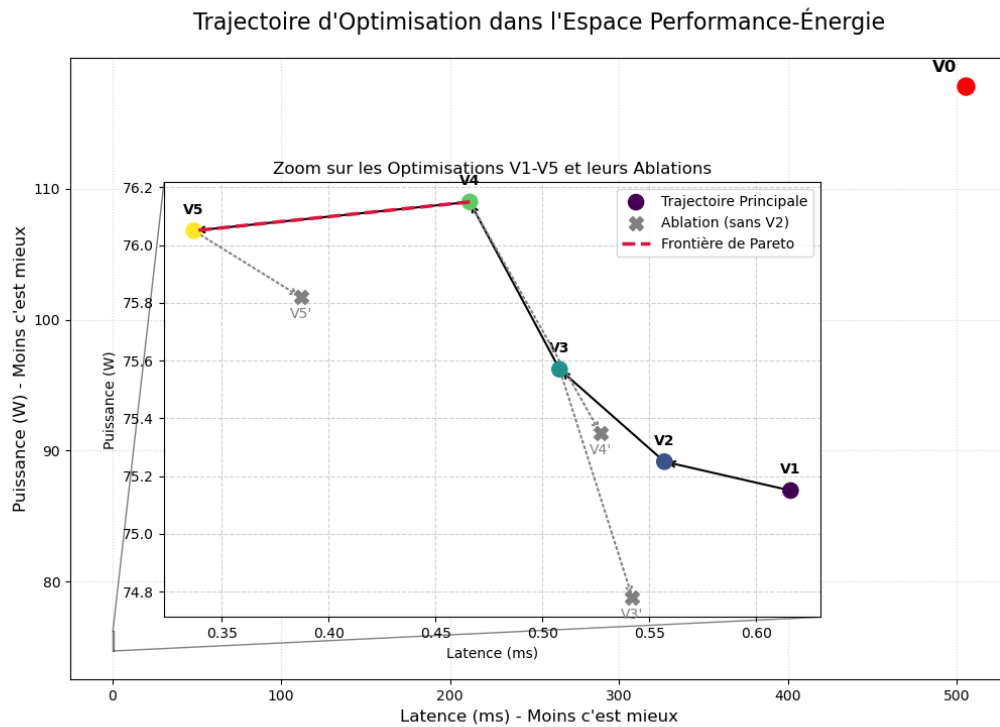


FIGURE 10 – Trajectoire d’optimisation complète. La trajectoire principale (cercles de couleur) montre la progression V1→V5. Les points d’ablation (croix grises) représentent les versions V3, V4 et V5 mais sans l’optimisation de coalescence (V2), démontrant l’impact de cette dernière.

La Figure 10 offre une synthèse visuelle de l’ensemble du travail d’optimisation mené durant ce stage...

Validation de la démarche et interdépendance des optimisations. La trajectoire V1 → V5 est une progression claire de l’optimisation. Chaque étape représente une intervention technique avec un gain mesurable. L’analyse révèle que ces gains sont non seulement significatifs, mais aussi cumulatifs.

Pour le prouver, des expériences d’ablation ont été menées (points V3’, V4’, V5’). Ces versions intègrent des optimisations avancées (comme le shuffle ou les Tensor Cores) tout en retirant volontairement une optimisation fondamentale précédente : la coalescence des accès mémoire (V2). Les flèches en pointillé sur le graphique visualisent le coût de cette omission : chaque version d’ablation est systématiquement moins performante que son homologue entièrement optimisé.

Cette analyse fine apporte une validation claire de la valeur de chaque brique d’optimisation. Elle permet à Earth Academy de justifier sa méthodologie en démontrant que la performance SOTA n’est pas due à une seule technique, mais le résultat d’une pile d’optimisations interdépendantes, où même les plus fondamentales restent critiques. Cela renforce la proposition de valeur en montrant une maîtrise complète de la chaîne de performance, de la base au sommet.

Implications pour le compromis performance-énergie. La frontière de Pareto, formée par les points V4 et V5, représente l’état de l’art atteignable et devient un outil de conseil stratégique. Elle permet de répondre à la question d’un client : "Quel est le meilleur résultat possible?". Le point V5 représente la latence minimale et donc l’efficacité énergétique maximale, définissant un objectif clair. Cette frontière constitue une référence objective et permet au client de prendre une décision éclairée sur l’investissement nécessaire pour atteindre le "meilleur compromis"

performance-énergie possible pour son application.

En conclusion, ce graphique valide la pertinence de l'offre de service envisagée par Earth Academy. Il prouve que l'optimisation technique est un levier majeur de la transition écologique numérique et fournit un support visuel et quantitatif pour structurer et vendre cette nouvelle expertise.

5.3 Validation de l'Approche In-Register

Validation du Potentiel sur un Micro-Benchmark Pour isoler et quantifier le bénéfice de la suppression de ce transit, un micro-benchmark a été développé, comparant directement l'approche standard à notre technique de permutation et réduction in-register. Les résultats, présentés dans le tableau 3, ont été validés sur deux architectures GPU distinctes pour en démontrer la robustesse.

TABLE 3 – Résultats du micro-benchmark sur un cas d'usage isolé (Matmul + Max). Notre technique démontre un potentiel d'accélération majeur en éliminant le transit par la mémoire partagée.

Kernel Version	Speedup (RTX 5080)	Speedup (H100)
Baseline (Shared Memory)	1.0x	1.0x
In-Register (Notre technique)	5.5x	6.1x

Ce gain de plus de 600% sur les deux plateformes valide l'hypothèse fondamentale : le transit par la mémoire partagée représente bien un coût de latence majeur et universel.

Intégration et Analyse sur un Kernel de Bout-en-Bout Pour valider l'impact de la technique in-register dans un contexte de bout-en-bout, une expérience contrôlée a été mise en place. L'objectif n'était pas de se mesurer au kernel V5, dont la complexité (pipelining, etc.) aurait introduit de multiples variables parasites, mais d'isoler **uniquement l'impact de la méthode de calcul du softmax**.

Pour ce faire, deux kernels dédiés ont été développés :

- Une **Baseline d'isolation**, un kernel de type FlashAttention fonctionnel mais volontairement simplifié (inférieur en performance à la version V1 de notre suite d'optimisation).
- Une version **In-Register**, identique en tout point à la baseline d'isolation, à la seule et unique exception du module de softmax, qui a été remplacé par notre technique in-register.

Cette approche garantit que toute différence de performance mesurée entre ces deux kernels est **directement et uniquement attribuable** à la technique de softmax elle-même. Les résultats de cette comparaison rigoureuse sont présentés dans le tableau 4.

TABLE 4 – Performance comparée sur les kernels d'isolation (RTX 5080, seq_len=1024, d_model=512).

Kernel Version	Latence (ms)
Baseline d'isolation (Softmax via Shared Memory)	3.53
Version In-Register (Softmax In-Register)	2.51
Speedup mesuré	1.40x

Ce résultat démontre un gain de performance de **40%** sur le kernel complet, prouvant l'efficacité de la technique in-register pour remplacer la boucle de softmax standard.

Il est également instructif de noter les défis rencontrés lors des tentatives d'intégration de cette technique au sein du kernel V5, notre version la plus optimisée. Une première tentative avait

abouti à un ralentissement (0.85x). L'analyse a révélé que ce n'était pas dû à la pression sur les registres, mais à l'extrême sensibilité du kernel V5 aux optimisations du compilateur. À ce niveau de performance, des facteurs comme l'alignement du code peuvent provoquer des variations de performance imprévisibles de l'ordre de 30%, masquant l'impact réel d'une modification locale.

La conclusion de cette phase est donc double. Premièrement, la technique in-register est **fondamentalement plus performante**, comme le prouve le gain de 1.40x de l'expérience d'isolation. Deuxièmement, l'intégration d'une nouvelle optimisation dans un kernel déjà à la limite du matériel est une tâche non-triviale qui expose des défis liés au comportement du compilateur, un domaine de recherche à part entière. Le stage s'est achevé avant que cette intégration complexe puisse être finalisée, mais la validation de la technique elle-même a été un succès.

6 Conclusion et Perspectives

6.1 Conclusion

Ce stage, initialement axé sur l'analyse et l'optimisation des compromis latence-énergie des kernels d'attention, a évolué d'une analyse de performance vers la conception d'une nouvelle technique de bas niveau. En suivant une démarche d'optimisation incrémentale rigoureuse, nous avons d'abord maîtrisé et implémenté les techniques de l'état de l'art, de la fusion de kernel à l'utilisation des Tensor Cores. Cette première phase a permis d'atteindre un niveau de performance SOTA, mais surtout, elle a mis en lumière un goulot d'étranglement persistant sur les architectures modernes, celui du cycle **Registre -> Mémoire Partagée -> Registre** imposé par les opérations post-multiplication.

La seconde phase du projet a été consacrée à la résolution de ce problème. Par une démarche de reverse-engineering de la structure des données WMMA, nous avons conçu et validé une nouvelle technique de permutation et de réduction s'exécutant entièrement dans les registres. Validée par une expérience d'isolation rigoureuse, cette approche a démontré un gain de performance de **1.40x** sur un kernel FlashAttention complet, confirmant son potentiel. L'ensemble de cette démarche et de ses résultats a été formalisé dans une pré-publication scientifique [6], constituant un des livrables majeurs de ce stage.

Ce projet apporte ainsi à Earth Academy une démonstration concrète et chiffrée, posant les bases solides de sa future offre de conseil en optimisation logicielle.

6.2 Limites et Perspectives

Ce travail, tout en validant une nouvelle approche, ouvre autant de questions qu'il n'en résout, définissant ainsi des axes clairs pour de futurs développements.

Limites Actuelles La principale limite de la technique développée est son manque de portabilité. L'algorithme de permutation est intimement lié à la structure des données non-documentée d'une architecture GPU spécifique (Blackwell dans notre cas) et d'une forme de matrice WMMA précise (16x16x16). Son application à d'autres architectures (Hopper, etc.) ou à d'autres tailles de matrices nécessiterait un nouvel effort de reverse-engineering et de conception.

Cependant, cette spécificité architecturale, loin d'être une simple contrainte, est en réalité la nouvelle frontière de l'optimisation SOTA. Des travaux récents sur *FlashAttention-4*, menés par des chercheurs de premier plan comme Tri Dao, confirment cette tendance. Leurs gains de performance reposent sur des optimisations de très bas niveau, si dépendantes du matériel qu'elles sont extrêmement difficiles à porter sur d'autres plateformes. Ce "fossé logiciel" (*software moat*) démontre que la valeur ne réside plus seulement dans l'algorithme, mais dans sa co-conception avec la micro-architecture. Notre travail s'inscrit directement dans ce paradigme : c'est un exemple concret de l'ingénierie de pointe nécessaire pour repousser les limites de la performance.

Perspectives Futures La véritable portée de ce travail réside dans la généralisation du motif d'optimisation qu'il établit : **Matmul** $\rightarrow$ **In-Register Row-wise Operation**. Le softmax n'est qu'une des nombreuses opérations qui suivent ce schéma. Plusieurs briques fondamentales des modèles d'IA pourraient bénéficier de cette approche :

- La **Layer Normalization** (ou RMSNorm), qui calcule une moyenne et une variance par ligne.
- Les opérations de **Max-Pooling** sur une dimension.
- Certaines **fonctions d'activation** ou de **calcul de coût** qui opèrent sur des lignes de données après une transformation linéaire.

L'exploration de ces cas d'usage constitue une piste de recherche prometteuse. Pour lever la barrière de la portabilité, une perspective à plus long terme serait le développement d'un outil d'analyse capable de découvrir programmatiquement le layout des données pour une architecture et une forme de WMMA données, puis de générer automatiquement le code de permutation optimal. Un tel outil transformerait cette technique artisanale de recherche en un puissant instrument d'ingénierie pour la prochaine génération de kernels IA.

Glossaire

Contexte et Concepts Fondamentaux (IA & Architecture)

Attention (Mécanisme d'Attention) Mécanisme au cœur des architectures Transformer. Il permet à un modèle de peser dynamiquement l'importance des différentes parties d'une séquence d'entrée pour effectuer une tâche, en calculant des scores de similarité entre des vecteurs *Query* (Q), *Key* (K) et en les utilisant pour pondérer des vecteurs *Value* (V).

Green AI Domaine de recherche visant à réduire l'empreinte environnementale (notamment énergétique) de l'intelligence artificielle. Ce rapport s'inscrit dans l'approche de *l'efficacité algorithmique* (maximiser la performance du matériel) plutôt que celle du *contrôle thermodynamique* (brider la consommation).

GPU (Graphics Processing Unit) Processeur massivement parallèle, optimisé pour le débit (*throughput*) sur des tâches simples et répétitives. Son architecture est fondamentale pour les calculs de l'IA moderne.

LLM (Large Language Model) Grand modèle de langage, un type de modèle d'IA basé sur l'architecture Transformer et entraîné sur de vastes corpus de texte pour comprendre et générer du langage humain.

Transformer Architecture de réseau de neurones qui repose principalement sur le mécanisme d'auto-attention. Elle est devenue le standard pour le traitement du langage naturel (NLP) et constitue la base de la plupart des LLM modernes.

Algorithmes et Processus Mathématiques

Softmax Fonction mathématique qui transforme un vecteur de nombres réels (les scores d'attention) en une distribution de probabilités. Chaque valeur en sortie est comprise entre 0 et 1, et la somme de toutes les valeurs est égale à 1.

Softmax Itératif (ou "Online") Algorithme utilisé dans FlashAttention qui calcule le softmax par blocs. Il maintient des statistiques (maximum et somme des exponentielles) et les met à jour correctivement à chaque nouvelle tuile traitée, évitant ainsi de devoir parcourir les données plusieurs fois (voir Annexe B pour la preuve).

Techniques d'Optimisation et Concepts GPU

Accès Coalescé Technique d'accès mémoire où les 32 threads d'un warp lisent des adresses contiguës, permettant au matériel de fusionner les requêtes en une seule transaction efficace.

- CUDA** Plateforme de programmation de NVIDIA permettant d’exploiter la puissance de calcul parallèle des GPU.
- Double Buffering** Technique masquant la latence mémoire en superposant le chargement des données de l’itération future avec le calcul sur les données de l’itération courante.
- In-Register** Qualifie une opération s’exécutant entièrement dans les registres du GPU, le niveau de mémoire le plus rapide, éliminant tout transit par la mémoire partagée.
- Kernel (CUDA)** Fonction exécutée en parallèle par les threads sur le GPU.
- Mémoire Partagée (Shared Memory)** Mémoire *on-chip* rapide, partagée par les threads d’un même bloc, utilisée pour la collaboration et la réutilisation des données.
- Permutation In-Register** La technique au cœur de la contribution de ce rapport. Elle consiste à réorganiser la disposition des données d’un fragment WMMA directement dans les registres à l’aide d’instructions `shuffle`, afin de les préparer pour une réduction parallèle sans passer par la mémoire partagée.
- Reverse-Engineering (du Layout WMMA)** Démarche expérimentale menée dans ce projet pour découvrir et cartographier la structure interne, non documentée, de la manière dont les données d’une matrice sont distribuées dans les registres des threads d’un warp par l’API WMMA.
- Shuffle (Primitives)** Instructions CUDA (`__shfl_sync`) permettant un échange de données direct et rapide de registre à registre entre les threads d’un même warp.
- Tensor Cores** Unités matérielles spécialisées dans les GPU NVIDIA pour accélérer massivement les opérations de multiplication-accumulation de matrices (MMA).
- Tiling (ou Tuilage)** Stratégie algorithmique fondamentale consistant à diviser un grand problème en sous-problèmes (tuiles) pouvant être traités dans la mémoire rapide du GPU pour maximiser la localité des données.
- Warp** Unité d’exécution de base d’un GPU NVIDIA, composée de 32 threads exécutant la même instruction en parallèle.
- WMMA (Warp-Matrix Multiply-Accumulate)** API C++ de CUDA qui fournit une interface pour programmer les Tensor Cores en manipulant des matrices sous forme de *fragments*.

Annexes

A Développement Durable et Responsabilité Sociétale

Étant donné que la mission de l’entreprise est intrinsèquement liée au développement durable, cette analyse se concentre sur la manière dont cette mission se traduit dans son secteur, son fonctionnement interne et le projet de stage lui-même.

A.1 Pressions sur l’Environnement

A.1.1 Du secteur d’activité de l’établissement. Le stage s’inscrit dans le secteur du numérique et plus spécifiquement de l’intelligence artificielle (IA), un domaine dont l’impact environnemental est un enjeu majeur et croissant. Les principales pressions exercées par ce secteur sont :

- **Consommation énergétique :** L’entraînement et l’inférence des grands modèles d’IA sont des processus extrêmement énergivores. Ils reposent sur des centres de données dont la consommation électrique mondiale est comparable à celle de pays entiers. Cet enjeu a plusieurs facettes : il concerne à la fois la consommation directe de ressources (électricité, eau pour le refroidissement) et les émissions de gaz à effet de serre, mais aussi l’impact physique des infrastructures sur les écosystèmes.

- **Fabrication du matériel** : La production de matériel de pointe (GPU, TPU) nécessite l'extraction de terres rares, une consommation d'eau et d'énergie significatives, et génère des déchets électroniques en fin de vie.
- Un risque bien connu dans ce domaine est l'effet rebond : en rendant une technologie plus efficace, on risque paradoxalement d'en augmenter l'usage, ce qui peut annuler une partie des bénéfices écologiques.

A.1.2 De l'établissement. Earth Academy, en tant qu'entreprise de conseil, a une empreinte directe plus limitée. Elle se concentre sur :

- **Impact numérique interne** : L'utilisation de services cloud pour la recherche (comme LambdaGPU pour ce stage) et les outils de communication.
- **Déplacements professionnels** : L'entreprise étant basée aux Pays-Bas, les déplacements pour rencontrer des clients internationaux représentent une source d'émissions.
- **Fonctionnement des bureaux** : Consommation énergétique, gestion des déchets, etc.

La spécificité de l'établissement est que son existence même est une réponse aux pressions de son secteur. Sa mission est de fournir l'expertise pour réduire l'impact de ses clients.

A.2 Politique, Stratégie et Actions pour Minimiser les Impacts

A.2.1 D'un point de vue du fonctionnement de l'établissement. Earth Academy applique en interne les principes qu'elle promeut à l'externe. Lors de mes échanges, il est apparu que l'entreprise a mis en place une politique interne stricte :

- **Énergie** : Les bureaux sont alimentés par un fournisseur d'électricité 100% renouvelable. Une politique d'"extinction systématique" est en place pour tous les appareils non essentiels.
- **Numérique responsable** : Le choix des fournisseurs de services cloud est un critère stratégique, privilégiant ceux qui affichent les meilleurs PUE (*Power Usage Effectiveness*) et qui s'engagent dans l'utilisation d'énergies renouvelables.
- **Déplacements** : Une politique "train d'abord" est appliquée pour tous les déplacements en Europe.

A.2.2 D'un point de vue du cœur de métier et de la stratégie. C'est ici que l'engagement d'Earth Academy est le plus fort. La durabilité n'est pas une contrainte ou une politique annexe, c'est son **unique produit**. La stratégie de l'entreprise n'a pas été "repensée" au prisme des enjeux environnementaux ; elle a été **fondée** sur eux. L'objectif est de dépasser les simples "éco-gestes" pour promouvoir l'éco-conception logicielle (*Green IT*). Le cœur de métier est de prouver, chiffres à l'appui, que la performance logicielle et la sobriété numérique sont non seulement compatibles, mais synergiques.

A.2.3 Lien entre ma mission de stage et la stratégie de l'établissement. Le lien est direct et total. Ma mission n'était pas un sujet de recherche isolé, mais le projet pilote d'une nouvelle offre stratégique pour Earth Academy.

- **Mission** : Mon mandat n'était pas simplement de "rendre un algorithme plus rapide", mais d'"analyser et optimiser la performance ET l'efficacité énergétique". La mesure de la consommation et le calcul de l'énergie par opération étaient des livrables aussi importants que la mesure de la latence.
- **Livrable stratégique** : Le graphique de la frontière de Pareto (Figure 10) est l'illustration parfaite de ce lien. Il n'est pas seulement un résultat scientifique ; c'est un prototype d'outil de conseil. Il permet de matérialiser pour un client le gain énergétique quantifiable associé à chaque intervention technique. Il transforme une expertise technique complexe en une proposition de valeur claire, alignée avec les objectifs de transition écologique.

Plus fondamentalement, la démarche de ce stage s’aligne sur le principe de **Green AI** [2, 3], qui promeut une IA plus soutenable et inclusive. Notre approche, centrée sur l’optimisation algorithmique profonde plutôt que sur le simple contrôle énergétique, a un double impact. D’une part, elle réduit l’énergie totale nécessaire pour une tâche en diminuant drastiquement son temps d’exécution. D’autre part, et c’est peut-être le plus crucial, elle démontre qu’il est possible d’atteindre des performances de pointe sur du matériel accessible.

Pour conclure, ce stage a été plus qu’une simple application technique : il a été une participation concrète à l’élaboration de la stratégie future de l’entreprise.

B Preuve Qualitative de la Sous-Optimalité des Réorganisations Intuitives

Une approche intuitive pour manipuler les données in-register après le calcul WMMA consisterait à restaurer entièrement un layout linéaire (c’est-à-dire, où tous les éléments d’une même ligne sont stockés de manière contiguë dans les registres d’un même thread, ou dans des threads adjacents). Cependant, l’analyse du mapping (Figure 8) révèle que les 16 éléments d’une ligne sont répartis sur 8 threads distincts.

Pour consolider tous les éléments d’une seule ligne sur un seul thread, il faudrait un schéma de communication complexe impliquant au moins 2 instructions `__shfl_sync` par ligne aux lanes extremales, et 4 pour les autres lanes (En exploitant un shuffle par bloc des sections à échanger), simplement pour rassembler les données. Cela donne une moyenne de > 3.87 shuffle pour la réorganisation des données, suivi d’un shuffle à `BLOCK_SIZE = 16` pour communiquer le maximum local, pour borne inférieure moyenne théorique > 4.87 shuffles contre 4 pour notre méthode.

De même, l’idée de laisser les frags organisés tel quel requièrerait à minima le même nombre de shuffle (même répartition des lignes oblige), mais introduirait une logique de de-swizzling si complexe (Cumule de bit shifting pour filtrer par colonne, et d’opérateurs ternaires imbriqués pour créer un masquage des colonnes, le tout dépendant de la lane, ce qui empêche un pré-calcul du compilateur) que son calcul suffirait à lui seul à couvrir la latence de quelques cycles ajoutée par le swapping de notre méthode.

Notre approche de permutation partielle est un compromis optimal : elle n’atteint pas une linéarité parfaite, mais elle regroupe les données en sous-groupes localement contigus, ce qui est **suffisant** pour permettre une réduction parallèle efficace avec un coût de réorganisation minimal (une seule opération de SWAP). L’objectif n’est pas la perfection structurelle, mais l’efficacité de l’opération qui suit.

C Preuve de l’Algorithme de Softmax Itératif

Cette section présente la preuve formelle de l’exactitude de l’algorithme de softmax calculé de manière itérative par blocs, tel qu’il est utilisé dans des algorithmes comme FlashAttention [4].

Notations Soit $S \in \mathbb{R}^{N \times N}$ la matrice de scores. Le calcul standard du softmax pour une ligne i est défini par :

$$m_i = \max_{1 \leq j \leq N} S_{ij} \quad (4)$$

$$l_i = \sum_{j=1}^N \exp(S_{ij} - m_i) \quad (5)$$

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{l_i} \quad (6)$$

Décomposons la matrice S en T blocs de colonnes de taille B : $S = [S^{(1)} S^{(2)} \dots S^{(T)}]$. Nous allons prouver par récurrence sur le nombre de blocs $t \in \{1, \dots, T\}$ que nous pouvons maintenir des statistiques $m_i(t)$ et $l_i(t)$ qui sont équivalentes à un calcul sur les $t \times B$ premières colonnes.

Proposition Après avoir traité t blocs, les statistiques $m_i(t)$ et $l_i(t)$ sont définies comme suit :

$$m_i(t) = \max_{1 \leq j \leq tB} S_{ij} \quad (7)$$

$$l_i(t) = \sum_{j=1}^{tB} \exp(S_{ij} - m_i(t)) \quad (8)$$

Preuve par récurrence

- **Cas de base (t=1)** : Pour le premier bloc, on calcule $m_i(1) = \max_{j \in S^{(1)}} S_{ij}$ et $l_i(1) = \sum_{j \in S^{(1)}} \exp(S_{ij} - m_i(1))$ directement sur la première tuile. La proposition est vérifiée.
- **Hypothèse de récurrence** : Supposons qu'au pas $t-1$, nous disposons des valeurs correctes $m_i(t-1)$ et $l_i(t-1)$ pour les $t-1$ premiers blocs.
- **Étape de récurrence (pas t)** : Nous traitons le bloc $S^{(t)}$.
 1. Le nouveau maximum global est :

$$m_i(t) = \max \left(\max_{1 \leq j \leq (t-1)B} S_{ij}, \max_{(t-1)B < j \leq tB} S_{ij} \right) = \max(m_i(t-1), m_i^{\text{local}}) \quad (9)$$

2. La nouvelle somme des exponentielles $l_i(t)$ est :

$$l_i(t) = \sum_{j=1}^{(t-1)B} \exp(S_{ij} - m_i(t)) + \sum_{j \in S^{(t)}} \exp(S_{ij} - m_i(t)) \quad (10)$$

En réécrivant le premier terme avec la propriété de l'exponentielle :

$$\sum_{j=1}^{(t-1)B} \exp(S_{ij} - m_i(t)) = \sum_{j=1}^{(t-1)B} \exp(S_{ij} - m_i(t-1) + m_i(t-1) - m_i(t)) \quad (11)$$

$$= \exp(m_i(t-1) - m_i(t)) \sum_{j=1}^{(t-1)B} \exp(S_{ij} - m_i(t-1)) \quad (12)$$

$$= l_i(t-1) \cdot \exp(m_i(t-1) - m_i(t)) \quad (13)$$

On obtient la formule de mise à jour :

$$l_i(t) = l_i(t-1) \cdot \exp(m_i(t-1) - m_i(t)) + \sum_{j \in S^{(t)}} \exp(S_{ij} - m_i(t)) \quad (14)$$

La proposition est donc vérifiée au rang t .

Par récurrence, les statistiques $m_i(T)$ et $l_i(T)$ calculées à la fin sont bien égales aux valeurs m_i et l_i de l'algorithme standard. **Q.E.D.**

D Code source des kernels du micro-benchmark

```
__global__ void Frag_standard_baseline(const __nv_bfloat16* A, const __nv_bfloat16* B, float* out) {
    __shared__ float sD[BLOCK_SIZE][BLOCK_SIZE];

    wmma::fragment<wmma::matrix_a, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, float> acc_frag;
```

```

wmma::load_matrix_sync(a_frag, A, BLOCK_SIZE);
wmma::load_matrix_sync(b_frag, B, BLOCK_SIZE);
wmma::fill_fragment(acc_frag, 0.0f);
wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);

wmma::store_matrix_sync(&sD[0][0], acc_frag, BLOCK_SIZE, wmma::mem_row_major);
__syncthreads();

int lane = threadIdx.x;
int my_row = lane % 16;
int start_col = (lane < 16) ? 0 : 8;

float local_max = -INFINITY;
for (int i = 0; i < 8; i++) {
    local_max = fmaxf(local_max, sD[my_row][start_col + i]);
}

unsigned mask = (1u << lane) | (1u << (lane ^ 16));
float partner_max = __shfl_xor_sync(mask, local_max, 16);
float final_row_max = fmaxf(local_max, partner_max);

if (lane < 16) {
    out[my_row] = final_row_max;
}
}

__global__ void Frag_swapped(const __nv_bfloat16* A, const __nv_bfloat16* B, float* out) {
    wmma::fragment<wmma::matrix_a, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, float> acc_frag;

    wmma::load_matrix_sync(a_frag, A, BLOCK_SIZE);
    wmma::load_matrix_sync(b_frag, B, BLOCK_SIZE);
    wmma::fill_fragment(acc_frag, 0.0f);
    wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);

    float temp_f = acc_frag.x[2];
    acc_frag.x[2] = acc_frag.x[4];
    acc_frag.x[4] = temp_f;
    temp_f = acc_frag.x[3];
    acc_frag.x[3] = acc_frag.x[5];
    acc_frag.x[5] = temp_f;

    float max_A = fmaxf(fmaxf(acc_frag.x[0], acc_frag.x[1]), fmaxf(acc_frag.x[2], acc_frag.x[3]));
    float max_B = fmaxf(fmaxf(acc_frag.x[4], acc_frag.x[5]), fmaxf(acc_frag.x[6], acc_frag.x[7]));

    unsigned int mask = 0xF << ((threadIdx.x / 4) * 4);
    max_A = fmaxf(max_A, __shfl_xor_sync(mask, max_A, 1));
    max_A = fmaxf(max_A, __shfl_xor_sync(mask, max_A, 2));
    max_B = fmaxf(max_B, __shfl_xor_sync(mask, max_B, 1));
    max_B = fmaxf(max_B, __shfl_xor_sync(mask, max_B, 2));

    if (threadIdx.x % 4 == 0) {
        int row_group = threadIdx.x / 4;
        out[row_group] = max_A;
        out[row_group + 8] = max_B;
    }
}

```


E Code Source du Kernel Final In-Register

```

1  __global__ void flash_attention_kernel(__nv_bfloat16* Q, __nv_bfloat16* K, __nv_bfloat16* V, __nv_bfloat16*
    ↪ out,
2      int seq_len, int d_model) {
3      // int bx = blockIdx.x;
4      // int by = blockIdx.y;
5      // int tx = threadIdx.x;
6      // int ty = threadIdx.y;
7      // int globRow = by*BLOCK_SIZE+ty;
8      // int globCol = bx*BLOCK_SIZE+tx;
9
10     if (blockIdx.x*BLOCK_SIZE >= d_model || blockIdx.y*BLOCK_SIZE >= seq_len) return;
11     unsigned int* vals;
12     float max_A;
13     float max_B;
14     float max_A_i = -INFINITY;
15     float max_B_i = -INFINITY;
16     float sum_A;
17     float sum_B;
18     float sum_A_i = 0.0f;
19     float sum_B_i = 0.0f;
20
21     __shared__ __nv_bfloat16 sQ_full[BLOCK_SIZE][PADDED_D];
22     __shared__ __nv_bfloat16 sK[2][BLOCK_SIZE][BLOCK_SIZE];
23     __shared__ __nv_bfloat16 sV[BLOCK_SIZE][BLOCK_SIZE];
24     __shared__ float sO_accum[32][8];
25
26     for(int i = 0; i < 256; i++) {
27         int flat_idx = threadIdx.x + i * 32;
28         int row = flat_idx / 512;
29         int col = flat_idx % 512;
30         if(row < 16) {
31             sQ_full[row][col] = Q[(blockIdx.y * 16 + row) * d_model + col];
32         }
33     }
34
35     for(int i = 0; i < 8; i++) {
36         sO_accum[threadIdx.x][i] = 0.0f;
37     }
38
39     // BOUCLE FLASH (externe) - Itère sur les blocs de K/V
40     for (int j = 0; j < seq_len / BLOCK_SIZE; ++j) {
41
42         wmma::fragment<wmma::matrix_a, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::row_major>
    ↪ q_frag;
43         wmma::fragment<wmma::matrix_b, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::col_major>
    ↪ k_frag;
44         wmma::fragment<wmma::accumulator, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, float> work_frag;
45         wmma::fill_fragment(work_frag, 0.0f);
46
47         // PROLOGUE sK[0]
48         for(int i = 0; i < 8; i++) {
49             int tile_idx = threadIdx.x + i * 32;
50             int row_in_tile = tile_idx / BLOCK_SIZE;
51             int col_in_tile = tile_idx % BLOCK_SIZE;
52             int global_K_row = j * BLOCK_SIZE + row_in_tile;
53             int global_K_col = 0 + col_in_tile;
54             sK[0][row_in_tile][col_in_tile] = K[global_K_row * d_model + global_K_col];
55         }
56
57         for (int p = 0; p < d_model / BLOCK_SIZE - 1; ++p) {
58             __syncthreads();
59
60             int current_buf_idx = p % 2;
61             int next_buf_idx = 1 - current_buf_idx;
62
63             for(int i = 0; i < 8; i++) {
64                 int tile_idx = threadIdx.x + i * 32;
65                 int row_in_tile = tile_idx / BLOCK_SIZE;
66                 int col_in_tile = tile_idx % BLOCK_SIZE;
67                 int global_K_row = j * BLOCK_SIZE + row_in_tile;

```

```

1      int global_K_col = (p + 1) * BLOCK_SIZE + col_in_tile;
2      sK[next_buf_idx][row_in_tile][col_in_tile] = K[global_K_row * d_model + global_K_col];
3  }
4
5
6      wmma::load_matrix_sync(q_frag, &sQ_full[0][p * BLOCK_SIZE], PADDED_D);
7      wmma::load_matrix_sync(k_frag, &sK[current_buf_idx][0][0], BLOCK_SIZE);
8      wmma::mma_sync(work_frag, q_frag, k_frag, work_frag);
9
10 }
11
12 int last_buf_idx = (d_model / BLOCK_SIZE - 1) % 2;
13 wmma::load_matrix_sync(q_frag, &sQ_full[0][(d_model / BLOCK_SIZE - 1) * BLOCK_SIZE], PADDED_D);
14 wmma::load_matrix_sync(k_frag, &sK[last_buf_idx][0][0], BLOCK_SIZE);
15 wmma::mma_sync(work_frag, q_frag, k_frag, work_frag);
16
17 for(int i = 0; i < 8; i++) {
18     work_frag.x[i] *= 1.0f / sqrtf((float)d_model);
19 }
20
21 float temp_f;
22
23 temp_f = work_frag.x[2];
24 work_frag.x[2] = work_frag.x[4];
25 work_frag.x[4] = temp_f;
26
27 temp_f = work_frag.x[3];
28 work_frag.x[3] = work_frag.x[5];
29 work_frag.x[5] = temp_f;
30
31 max_A = fmaxf(fmaxf(work_frag.x[0], work_frag.x[1]), fmaxf(work_frag.x[2], work_frag.x[3]));
32 max_B = fmaxf(fmaxf(work_frag.x[4], work_frag.x[5]), fmaxf(work_frag.x[6], work_frag.x[7]));
33
34 float partner_max_A = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), max_A, 1);
35 float partner_max_B = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), max_B, 1);
36
37 max_A = fmaxf(max_A, partner_max_A);
38 max_B = fmaxf(max_B, partner_max_B);
39
40 partner_max_A = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), max_A, 2);
41 partner_max_B = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), max_B, 2);
42
43 max_A = fmaxf(max_A, partner_max_A);
44 max_B = fmaxf(max_B, partner_max_B);
45
46 sum_A = 0.0f;
47 sum_B = 0.0f;
48 for(int i = 0; i < 4; ++i) {
49     work_frag.x[i] = expf(work_frag.x[i] - max_A);
50     sum_A += work_frag.x[i];
51 }
52 for(int i = 4; i < 8; ++i) {
53     work_frag.x[i] = expf(work_frag.x[i] - max_B);
54     sum_B += work_frag.x[i];
55 }
56
57 partner_max_A = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), sum_A, 1);
58 partner_max_B = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), sum_B, 1);
59
60 sum_A += partner_max_A;
61 sum_B += partner_max_B;
62
63 partner_max_A = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), sum_A, 2);
64 partner_max_B = __shfl_xor_sync(0xF << ((threadIdx.x / 4) * 4), sum_B, 2);
65
66 sum_A += partner_max_A;
67 sum_B += partner_max_B;
68
69 float old_sum_A_i = sum_A_i;
70 float old_sum_B_i = sum_B_i;
71
72 sum_A_i = old_sum_A_i * expf(max_A_i - fmax(max_A_i, max_A)) + sum_A * expf(max_A - fmax(max_A_i,
73     ↪ max_A));
74 sum_B_i = old_sum_B_i * expf(max_B_i - fmax(max_B_i, max_B)) + sum_B * expf(max_B - fmax(max_B_i,
75     ↪ max_B));

```

```

1
2
3      wmma::fragment<wmma::matrix_a, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::row_major>
      ↪ p_frag;
4
5      for(int i = 0; i < 4; ++i) {
6          p_frag.x[i] = __float2bfloat16(work_frag.x[i] / sum_A);
7      }
8      for(int i = 4; i < 8; ++i) {
9          p_frag.x[i] = __float2bfloat16(work_frag.x[i] / sum_B);
10     }
11
12     vals = reinterpret_cast<unsigned int*>(p_frag.x); //Un swap pour 2 valeurs: La reinterprétation
      ↪ prend 2 bits soit deux bfloat16, ce qui permet un swap en une seule instruction. Y'a pas de
      ↪ petites économies :)
13     unsigned int temp = vals[1];
14     vals[1] = vals[2];
15     vals[2] = temp;
16
17     wmma::fragment<wmma::matrix_b, BLOCK_SIZE, BLOCK_SIZE, BLOCK_SIZE, __nv_bfloat16, wmma::row_major>
      ↪ v_frag;
18     wmma::fill_fragment(work_frag, 0.0f);
19
20     for(int i = 0; i < 8; i++) {
21         int tile_idx = threadIdx.x + i * 32;
22         int row_in_tile = tile_idx / BLOCK_SIZE;
23         int col_in_tile = tile_idx % BLOCK_SIZE;
24
25         int global_V_row = j * BLOCK_SIZE + row_in_tile;
26
27         int global_V_col = blockIdx.x * BLOCK_SIZE + col_in_tile;
28
29         if (global_V_row < seq_len && global_V_col < d_model) {
30             sV[row_in_tile][col_in_tile] = V[global_V_row * d_model + global_V_col];
31         } else {
32             sV[row_in_tile][col_in_tile] = __float2bfloat16(0.0f);
33         }
34     }
35     __syncthreads();
36
37     wmma::load_matrix_sync(v_frag, &sV[0][0], 16);
38     wmma::mma_sync(work_frag, p_frag, v_frag, work_frag);
39
40     temp_f = work_frag.x[2];
41     work_frag.x[2] = work_frag.x[4];
42     work_frag.x[4] = temp_f;
43
44     temp_f = work_frag.x[3];
45     work_frag.x[3] = work_frag.x[5];
46     work_frag.x[5] = temp_f;
47
48     float m_new_A = fmax(max_A_i, max_A);
49     float m_new_B = fmax(max_B_i, max_B);
50
51     for(int i = 0; i < 4; ++i) {
52         s_0_accum[threadIdx.x][i] = s_0_accum[threadIdx.x][i] * (old_sum_A_i / sum_A_i) * expf(max_A_i
      ↪ - m_new_A)
53         + work_frag.x[i] * (sum_A / sum_A_i) * expf(max_A - m_new_A);
54     }
55
56     for(int i = 4; i < 8; ++i) {
57         s_0_accum[threadIdx.x][i] = s_0_accum[threadIdx.x][i] * (old_sum_B_i / sum_B_i) * expf(max_B_i
      ↪ - m_new_B)
58         + work_frag.x[i] * (sum_B / sum_B_i) * expf(max_B - m_new_B);
59     }
60
61     max_A_i = fmax(max_A_i, max_A);
62     max_B_i = fmax(max_B_i, max_B);
63
64     __syncthreads();
65 }
66
67 float temp_f = s_0_accum[threadIdx.x][2];
68 s_0_accum[threadIdx.x][2] = s_0_accum[threadIdx.x][4];
69 s_0_accum[threadIdx.x][4] = temp_f;
70

```

```
1
2     temp_f = s_0_accum[threadIdx.x][3];
3     s_0_accum[threadIdx.x][3] = s_0_accum[threadIdx.x][5];
4     s_0_accum[threadIdx.x][5] = temp_f;
5
6     // Écriture avec les formules de mapping inverse
7     for(int i = 0; i < 8; i++) {
8         out[(blockIdx.y * 16 + (threadIdx.x / 4) + (((i >> 1) & 1) * 8)) * d_model + blockIdx.x * 16 +
            ↪ ((threadIdx.x % 4) * 2) + (i % 2) + (((i >> 2) & 1) * 8)] =
            ↪ __float2bfloat16(s_0_accum[threadIdx.x][i]);
9     }
10 }
```

Références

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv :1706.03762*, 2017.
- [2] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv :2104.10350*, 2021.
- [3] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green AI. *Communications of the ACM*.
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention-2 : Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv :2205.14135*, 2022.
- [5] Tri Dao. Flashattention-2 : Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv :2307.08691*, 2023.
- [6] Tommy Morales. Accelerating fused wmma operations via in-register data permutation. 2025. HAL Preprint.
- [7] NVIDIA Corporation. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2025.
- [8] NVIDIA Corporation. TENSOR CORE DL PERFORMANCE GUIDE. <https://developer.download.nvidia.com/video/gputechconf/gtc/2019/presentation/s9926-tensor-core-performance-the-ultimate-guide.pdf>, 2019.